

UNIVERSITY OF BELGRADE
SCHOOL OF ELECTRICAL ENGINEERING

MASTER'S THESIS

Efficient Hardware Implementation of a
Mixed-Radix Fast Fourier Transform
Algorithm

supervisor:
dr Vladimir Petrović, assistant professor

student:
Lazar Dobrić
index: 2024/3087

Belgrade, august 2026.

Abstract

U ovom radu su predstavljene teorijske osnove LU dekompozicije matrica, analizirani su numerički algoritmi potrebni za njenu implementaciju, softverski je simulirana protočna hardverska arhitektura za dekompoziciju i realizovan je njen hardver na FPGA (*Field Programmable Gate Array*) čipu.

Keywords: LU dekompozicija, protočna arhitektura, FPGA

Contents

1	Introduction	2
2	Theoretical background	4
2.1	OFDM	4
2.1.1	5G NR standard physical layer	5
2.1.2	FFT sizes and radices required by 5G NR	6
2.2	FFT	8
2.2.1	FT	8
2.2.2	FFT	9
2.2.3	Mixed-radix FFT	16
2.2.4	Digit inversion	19
2.3	Inverse FFT	21
2.4	Fixed-point number representation	22
3	Hardware Architecture	25
3.1	Streaming Mixed-Radix FFT Architecture	26
3.1.1	SDF Architecture	26
3.2	Microarchitecture	29
3.2.1	Radix-2 Stage	31
3.2.2	Radix-2/3 Stage	33
3.2.3	Radix-2/3/5 Stage	35
3.2.4	Twiddle Factor Unit	37
3.2.5	Rotator	38
3.2.6	Scaling and Rounding	39
3.2.7	FIFOs	41
3.2.8	Phase and Delay Counters	41
3.2.9	Control Logic	42
3.2.10	Flow Control	43
3.2.11	Top-level control	44
4	Software Reference Model	47

Chapter 1

Introduction

Rešavanje sistema linearnih jednačina je srž jako velikog broja inženjerskih problema. Simulacije električnih kola i elektromagnetskih polja, adaptivno formiranje snopa i kriptovanje podataka skrivenih u slikama nakon steganografije su samo neki od njih. Problem rešavanja sistema se svodi na problem nalaženja inverza matrice sistema koji je složenosti $O(n^3)$ i supstituciju slobodnih članova. Ukoliko se koeficijenti jednačina sistema ne menjaju, ali se slobodni članovi menjaju, moguće je problem supstitucije slobodnih članova svesti sa složenosti $O(n^3)$ na dva problema složenosti $O(n^2)$, što zahteva mnogo manji broj aritmetičkih operacija. U ovom radu će biti analiziran način da se ovo izvede korišćenjem LU dekompozicije matrica. Već dugi niz godina postoje vrlo efikasne softverske implementacije LU dekompozicije visokih performansi, kako na jednoprocesorskim, tako i na višeprocesorskim sistemima. Usled popularizacije SDR-a (*Software Defined Radio*, softverski definisani radio), sve više su potrebne implementacije algoritama linearne algebre pogodne za hardversku realizaciju na rekonfigurabilnim platformama kao što je FPGA.

Cilj ovog rada je da objasni numeričke algoritme potrebne za LU dekompoziciju, predstavi i analizira protočne arhitekture za LU dekompoziciju i pokaže rezultate hardverske realizacije arhitekture za LU dekompoziciju.

Druga glava rada se bavi teorijskom osnovom rešavanja sistema linearnih jednačina, LU dekompozicije i numeričkom analizom algoritama potrebnih za hardversku realizaciju.

Treća glava opisuje postojeće hardverske arhitekture za LU dekompoziciju kao i modifikovanu arhitekturu koja se u nastavku rada simulira i realizuje hardverski. Pored toga, bavi se i hardverskom arhitekturom metoda za nalaženje recipročne vrednosti.

U četvrtoj glavi su predstavljeni rezultati simulacija softverskih modela arhitektura, simuliranih u Python programskom jeziku, dok se peta glava

bavi samom hardverskom realizacijom arhitektura.

Glava šest predstavlja neka potencijalna unapređenja hardverski realizovanih arhitektura.

Na kraju rada se nalazi zaključak.

Chapter 2

Theoretical background

In this chapter, the theoretical background necessary for understanding the architecture and implementation of the mixed-radix FFT algorithm will be discussed.

2.1 OFDM

Modern communication systems like 4G LTE, 5G or WiFi face a number of problems with an accent on a few major ones: - high data rates, - limited spectral bandwidth, - urban environments, - users moving through space.

To meet the requirements of a modern life they have to provide through wireless communication extremely high data rates in demanding urban environments full of radio-reflective materials and densely packed buildings. Modern users entail the ability to use their phone while walking down a street or riding on a bus.

All these requirements manifest though issues when designing such a system. High data rates within a limited spectral bandwidth require high spectral efficiency. Wave propagation in concrete environments and moving users introduce multipath propagation, frequency selective fading and Doppler effect.

The issue arises when, given a bandwidth throughout which a frequency selective fading is present, a modulation scheme that modulates symbols at the symbol rate $1/T$ that matches the bandwidth W is used. Different frequency components of a received symbol will suffer different channel fading when the signal passes through the channel. To suppress this, one could try Frequency Division Multiplexing (FDM) where the whole bandwidth W is split into N subchannels that are being used in parallel. If each such subchannel is narrow enough, the channel frequency response is essentially constant across it which solves the fading issue. But splitting a single channel

into multiple narrower subchannels requires additional guard bands between them to ensure there's no overlapping so inter-carrier interference is kept low enough. Because of guard bands and non-overlapping, spectral efficiency drops from the original single channel configuration.

citiraj proakisa ispod.

5G and modern WiFi systems aim to solve these issues by using Orthogonal Frequency Division Multiplexing (OFDM). Single band of width W is split into N subchannels of width W/N . By selecting the symbol rate $1/T$ in each of the subchannels to be equal to the frequency separation Δf of the adjacent subcarriers, the subcarriers are orthogonal over the symbol interval T . Now if N is chosen such that $\Delta f = W/N$ is small enough, channel frequency response $C(f)$ is essentially constant across each of the subchannels. This way it's possible to yield transmission rates close to channel capacity and minimize the effect of frequency selective fading.

One could remember that the Discrete Fourier Transform (DFT) essentially calculates the coefficients of the linear combination of complex exponentials that are orthogonal over the interval of length N . So, here the orthogonality of subcarriers allows for the efficient way of modulation and demodulation using Discrete Fourier Transform (DFT) by using calculation algorithm called Fast Fourier Transform (FFT).

2.1.1 5G NR standard physical layer

The physical layer of the 5G NR (*New Radio*) standard is defined by 3GPP in [3]. It specifies CP-OFDM (OFDM with a cyclic prefix) as the waveform in both downlink and uplink, with QPSK, 16-QAM, 64-QAM and 256-QAM used to modulate the individual subcarriers. Amongst all the physical layer parameters, the most important ones for the OFDM modulator and demodulator are the subcarrier spacing Δf and the number of used subcarriers, since these two directly determine the required DFT size and the sampling rate of the system.

Unlike 4G LTE, which uses a single fixed subcarrier spacing of 15 kHz, 5G NR defines a scalable numerology. The numerology is selected by the parameter μ and the subcarrier spacing is given by

$$\Delta f = 2^\mu \cdot 15 \text{ kHz}, \quad \mu \in \{0, 1, 2, 3, 4\}. \quad (2.1)$$

The corresponding OFDM symbol duration is $T = 1/\Delta f$. Narrow subcarrier spacings (15 and 30 kHz) are used in the FR1 frequency range (sub-6 GHz bands) where large cells and long channel delay spreads require a long cyclic prefix, while wide spacings (120 kHz and above) are used in the FR2 range

(millimeter-wave bands) where robustness to phase noise and Doppler shift is needed. The supported numerologies are summarized in Table 2.1.

Table 2.1: 5G NR numerologies [3].

μ	Δf [kHz]	Symbol duration T [μs]	Frequency range
0	15	66.67	FR1
1	30	33.33	FR1
2	60	16.67	FR1, FR2
3	120	8.33	FR2
4	240	4.17	FR2 (synchronization only)

In the frequency domain, subcarriers are grouped into resource blocks (RB), where one RB consists of 12 consecutive subcarriers. A carrier may contain at most 275 RBs, i.e. 3300 active subcarriers. The number of RBs for each combination of channel bandwidth and subcarrier spacing is specified in [4]. For example, a 100 MHz carrier with $\Delta f = 30$ kHz contains 273 RBs, which amounts to 3276 active subcarriers.

2.1.2 FFT sizes and radices required by 5G NR

OFDM modulation and demodulation are implemented as an IDFT and DFT, calculated using the FFT algorithm. The transform size N must be larger than or equal to the number of active subcarriers (if larger, the remaining inputs are set to zero and act as guard bands). The choice of N also determines the sampling rate of the baseband signal,

$$f_s = N \cdot \Delta f. \quad (2.2)$$

The standard itself is defined around a reference transform size of $N = 4096$: the basic time unit of NR is $T_c = 1/(\Delta f_{max} \cdot N_f)$, where $\Delta f_{max} = 480$ kHz and $N_f = 4096$ [3]. Transform sizes for other configurations are an implementation choice, and are typically chosen as the smallest size that fits all active subcarriers. Table 2.2 lists transform sizes and sampling rates for several typical NR carrier configurations.

Most of the required transform sizes are powers of two (512, 1024, 2048, 4096), which can be calculated using only radix-2 stages (or, equivalently, radix-4 and radix-8 stages). However, for some configurations the number of active subcarriers falls just above a power of two, and using the next power of two would be wasteful. For a 15 MHz carrier with $\Delta f = 15$ kHz there are 948 active subcarriers, so $N = 1024$ is too small, while $N = 2048$ would

Table 2.2: Typical FFT sizes for 5G NR carrier configurations [4].

Bandwidth [MHz]	Δf [kHz]	Active subcarriers	N	f_s [MHz]
5	15	300	512	7.68
10	15	624	1024	15.36
15	15	948	1536	23.04
20	15	1272	2048	30.72
50	30	1584	2048	61.44
80	30	2604	3072	92.16
100	30	3276	4096	122.88
400	120	3168	4096	491.52

double the sampling rate to 30.72 MHz. Instead, the size $N = 1536 = 3 \cdot 2^9$ is used, which reduces the sampling rate by 25% compared to $N = 2048$. Similarly, $N = 3072 = 3 \cdot 2^{10}$ is used for an 80 MHz carrier with $\Delta f = 30$ kHz instead of $N = 4096$. A lower sampling rate directly relaxes the throughput requirements of the FFT hardware and of the digital front-end that follows it. Supporting these sizes requires one radix-3 stage in addition to the radix-2 stages.

An even stronger requirement for mixed-radix processing comes from the NR uplink. Besides CP-OFDM, the uplink optionally uses DFT-s-OFDM (*DFT-spread OFDM*, also called transform precoding), where the modulation symbols are first passed through a DFT before being mapped onto the subcarriers of the OFDM modulator. This spreading lowers the PAPR (*Peak-to-Average Power Ratio*) of the transmitted signal, which improves the power amplifier efficiency of coverage-limited user devices. The size of the spreading DFT equals the number of allocated subcarriers,

$$M = 12 \cdot M_{RB}, \quad (2.3)$$

where M_{RB} is the number of allocated resource blocks. To guarantee that this DFT can always be calculated efficiently, the standard [3] restricts the allocation to values of the form

$$M_{RB} = 2^{\alpha_2} \cdot 3^{\alpha_3} \cdot 5^{\alpha_5}, \quad (2.4)$$

where α_2 , α_3 and α_5 are non-negative integers. There are exactly 53 such values of M_{RB} not exceeding the maximum of 275 RBs, listed in Table 2.3. The corresponding DFT sizes range from $M = 12$ (1 RB) to $M = 3240$ (270 RBs), and every one of them factorizes exclusively into the factors 2, 3 and 5.

Table 2.3: The 53 allowed values of M_{RB} for DFT-s-OFDM in 5G NR [3].

1	2	3	4	5	6	8	9	10	12
15	16	18	20	24	25	27	30	32	36
40	45	48	50	54	60	64	72	75	80
81	90	96	100	108	120	125	128	135	144
150	160	162	180	192	200	216	225	240	243
250	256	270							

Therefore, an FFT processor intended for a 5G NR modem must support all transform sizes of the form

$$N = 2^a \cdot 3^b \cdot 5^c, \quad (2.5)$$

covering both the OFDM modulator sizes $\{512, 1024, 1536, 2048, 3072, 4096\}$ and the 53 transform precoding sizes $M = 12 \cdot M_{RB}$. This is exactly the motivation for the mixed-radix FFT architecture presented in this work: a single hardware architecture combining radix-2, radix-3 and radix-5 butterfly stages can cover all transform sizes required by the 5G NR physical layer.

All 53 FFT sizes for the FFT done before the OFDM modulator are displayed in Table 2.4.

Table 2.4: The 53 transform precoding sizes $M = 12 \cdot M_{RB}$ for DFT-s-OFDM in 5G NR [3].

12	24	36	48	60	72	96	108	120	144
180	192	216	240	288	300	324	360	384	432
480	540	576	600	648	720	768	864	900	960
972	1080	1152	1200	1296	1440	1500	1536	1620	1728
1800	1920	1944	2160	2304	2400	2592	2700	2880	2916
3000	3072	3240							

2.2 FFT

2.2.1 FT

The Fourier Transform $F(\omega)$ is a mathematical transformation, expressed as (2.6), that transforms a continuous complex signal $f(t)$ from time to frequency domain. It is a tool for analysis of frequency content of a signal.

$$F(\omega) = \int_{-\infty}^{\infty} f(t)e^{-j\omega t} dt \quad (2.6)$$

Digital systems process discrete signals, so a more useful tool for them is DFT, expressed as (2.7), that transforms a discrete complex signal $x[n]$ ($n \in \{0, 1, \dots, N-1\}$) from time to frequency domain, where $X[k]$ ($k \in \{0, 1, \dots, N-1\}$) are the coefficients that stand with the basis complex exponentials of the DFT. N is the number of samples in the signal.

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j\frac{2\pi}{N}nk} \quad (2.7)$$

Complex factors $e^{-j\frac{2\pi}{N}nk}$ are usually called twiddle factors. They are commonly denoted as W_N^{nk} , so that:

$$W_N^{nk} = e^{-j\frac{2\pi}{N}nk}. \quad (2.8)$$

So, DFT can be expressed as:

$$X[k] = \sum_{n=0}^{N-1} x[n]W_N^{nk}. \quad (2.9)$$

Calculation of DFT by its definition is $O(N^2)$ complexity. This is because for each k from 0 to $N-1$ there are N complex multiplications and $N-1$ complex additions. This can be reduced to $O(N \log N)$ complexity by using Fast Fourier Transform (FFT) algorithms.

2.2.2 FFT

To reduce the complexity of DFT calculation, a Fast Fourier Transform (FFT) algorithms are used. Many different FFT algorithms exist, but the most common one is the Cooley-Tukey algorithm.

The Cooley-Tukey algorithm is based on a fact that calculations of some terms in DFT can be reused for calculations of other terms by doing a rearrangement of the terms. This way, the number of calculations can be reduced. The rearrangement can be done on input samples or on output samples. This is called decimation in time (DIT) or decimation in frequency (DIF) respectively. Further, we will explain the algorithm that uses DIF for the rearrangement.

A DFT length N can be factorized into $N = N_1 N_2$ where N_1 and N_2 are natural numbers. Thus, a DFT of length N can be calculated as N_1 DFTs of

length N_2 . If $N_1 = N_2$, the DFT of length N can be calculated as two DFTs of length $N/2$. This is a special case known as radix-2 FFT.

The described can be mathematically expressed as:

$$\begin{aligned}
X[k] &= \sum_{n=0}^{N-1} x[n] \cdot W_N^{nk} \\
&= \sum_{n=0}^{N_1-1} x[n] \cdot W_N^{nk} + \sum_{n=N_1}^{N-1} x[n] \cdot W_N^{nk} \\
&= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{nk} + \sum_{n=N/2}^{N-1} x[n] \cdot W_N^{nk} \\
&= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{nk} + \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{(n+N/2)k} \\
&= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{nk} + W_N^{N/2k} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{nk}.
\end{aligned} \tag{2.10}$$

Twiddle factors satisfy the identity described in (2.11).

$$W_N^{2nk} = e^{-j\frac{2\pi}{N}2nk} = e^{-j\frac{2\pi}{N/2}nk} = W_{N/2}^{nk}. \tag{2.11}$$

Thus, by using the identity described in (2.11) and substituting the $k = 2m$ ($m \in \{0, 1, \dots, N/2 - 1\}$) we calculate the even indexed output samples $X[2m]$ as:

$$\begin{aligned}
X[2m] &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot 2m} + W_N^{N/2 \cdot 2m} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{n \cdot 2m} \\
&= \sum_{n=0}^{N/2-1} x[n] \cdot W_{N/2}^{nm} + W_N^{Nm} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_{N/2}^{nm} \\
&= \sum_{n=0}^{N/2-1} (x[n] \cdot W_{N/2}^{nm} + W_N^{Nm} \cdot x[n + N/2] \cdot W_{N/2}^{nm}) \\
&= \sum_{n=0}^{N/2-1} (x[n] + x[n + N/2]) \cdot W_{N/2}^{nm}, \quad W_N^{Nm} = 1.
\end{aligned} \tag{2.12}$$

If we denote $G(m) = X[2m]$, then $G(m)$ can be viewed as a DFT of of sequence $g(n) = x[n] + x[n + N/2]$ of length $N/2$.

Similarly, if we substitute the $k = 2m + 1$ ($m \in \{0, 1, \dots, N/2 - 1\}$) we calculate the odd indexed output samples $X[2m+1]$ ($m \in \{0, 1, \dots, N/2 - 1\}$) as:

$$\begin{aligned}
X[2m+1] &= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot (2m+1)} + W_N^{N/2 \cdot (2m+1)} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{n \cdot (2m+1)} \\
&= \sum_{n=0}^{N/2-1} x[n] \cdot W_N^{n \cdot 2m} \cdot W_N^n + W_N^{N/2 \cdot 2m} \cdot W_N^{N/2} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_N^{n \cdot 2m} \cdot W_N^n \\
&= \sum_{n=0}^{N/2-1} x[n] \cdot W_{N/2}^{n \cdot m} \cdot W_N^n + W_N^{N/2 \cdot m} \cdot W_N^{N/2} \cdot \sum_{n=0}^{N/2-1} x[n + N/2] \cdot W_{N/2}^{n \cdot m} \cdot W_N^n \\
&= \sum_{n=0}^{N/2-1} ((x[n] - x[n + N/2]) \cdot W_N^n) \cdot W_{N/2}^{n \cdot m}, \quad W_N^{N/2} = -1.
\end{aligned} \tag{2.13}$$

If we denote $H(m) = X[2m+1]$, then $H(m)$ can be viewed as a DFT of of sequence $h(n) = (x[n] - x[n + N/2]) \cdot W_N^n$ of length $N/2$.

The equations (2.12) and (2.13) are graphically represented in Figure 2.1 for $N = 8$.

If the length of the DFT is a power of two, the algorithm can be applied recursively until the base case of length 2 is reached.

Then, the diagram in Figure 2.1 can be presented as in Figure 2.2.

A base element of this diagram is called a butterfly. It is a basic building block of the FFT algorithm and it essentially calculates the DFT of two samples. It is represented in Figure 2.3.

A line with symbol W_N^t above it represents a complex multiplication and nodes with two inputs and an output represents a complex addition. By observing the diagram in Figure 2.3 it can be seen that each stage of the FFT requires N complex additions to combine even and odd samples and $N/2$ complex multiplications to multiply half of the samples by the twiddle factors. Radix-2 FFT requires $\log N$ stages, so thus complexity of the FFT algorithm is $O(N \log N)$.

The final radix-2 FFT algorithm can be expressed in the next two equations:

$$X[k] = \sum_{m=0}^{N/2-1} (x[2m] + x[2m+1] \cdot W_N^k) \cdot W_{N/2}^{mk} \tag{2.14}$$

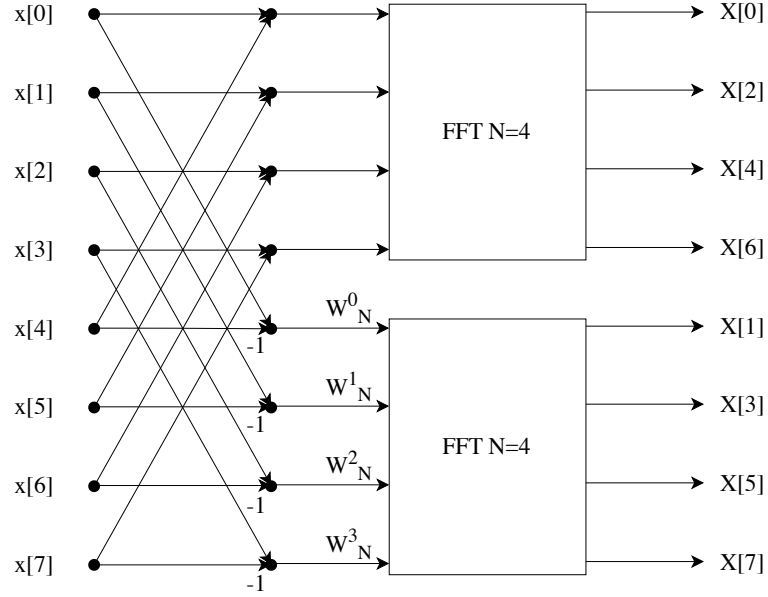


Figure 2.1: Graphical representation of the equations (2.12) and (2.13) for $N = 8$.

$$X[k + N/2] = \sum_{m=0}^{N/2-1} (x[2m] - x[2m + 1] \cdot W_N^k) \cdot W_{N/2}^{mk} \quad (2.15)$$

One could try changing N_1 to 3 and that would lead to a radix-3 FFT algorithm. Then, the (2.10) equation would change to:

$$\begin{aligned} X[k] &= \sum_{n=0}^{N-1} x[n] \cdot W_N^{nk} \\ &= \sum_{n=0}^{N/3-1} x[n] \cdot W_N^{nk} + \sum_{n=N/3}^{2N/3-1} x[n] \cdot W_N^{nk} + \sum_{n=2N/3}^{N-1} x[n] \cdot W_N^{nk} \end{aligned} \quad (2.16)$$

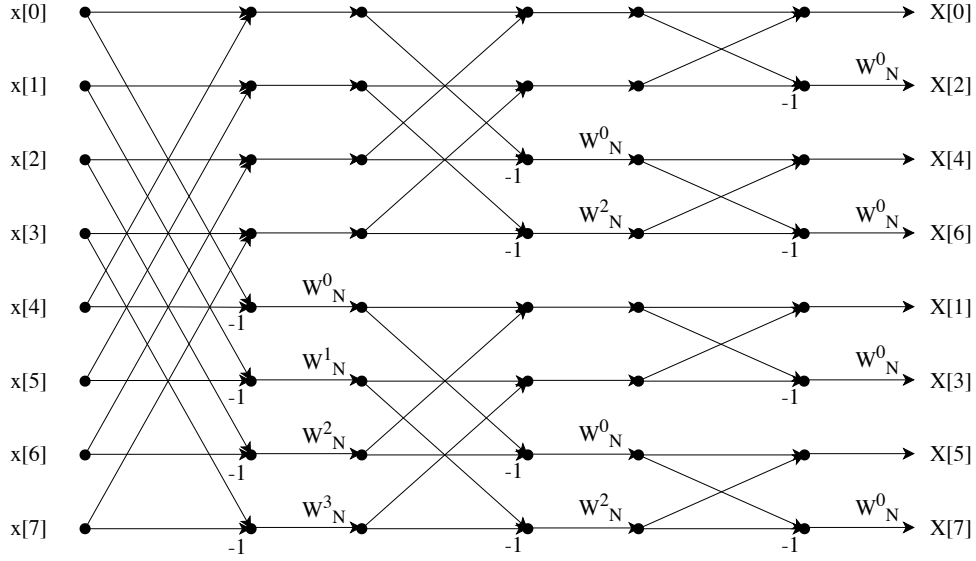


Figure 2.2: Graphical representation of the equations (2.12) and (2.13) for $N = 8$.

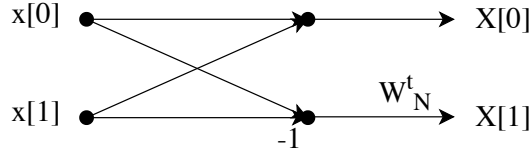


Figure 2.3: Graphical representation of the butterfly for $N = 8$.

then by defining $k = 3m$ and substituting it in the equation (2.16) we get:

$$\begin{aligned}
X[3m] &= \sum_{n=0}^{N/3-1} x[n] \cdot W_N^{n3m} + \sum_{n=N/3}^{2N/3-1} x[n] \cdot W_N^{n3m} + \sum_{n=2N/3}^{N-1} x[n] \cdot W_N^{n3m} \\
&= \sum_{n=0}^{N/3-1} x[n] \cdot W_{N/3}^{nm} + \sum_{n=0}^{N/3-1} x[n + N/3] \cdot W_{N/3}^{(n+N/3)m} + \sum_{n=0}^{N/3-1} x[n + 2N/3] \cdot W_{N/3}^{(n+2N/3)m} \\
&= \sum_{n=0}^{N/3-1} x[n] \cdot W_{N/3}^{nm} + W_N^{Nm} \sum_{n=0}^{N/3-1} x[n + N/3] \cdot W_{N/3}^{nm} + W_N^{2Nm} \sum_{n=0}^{N/3-1} x[n + 2N/3] \cdot W_{N/3}^{nm} \\
&= \sum_{n=0}^{N/3-1} ((x[n] + x[n + N/3] + x[n + 2N/3]) \cdot W_{N/3}^{nm}).
\end{aligned} \tag{2.17}$$

If we plug in $k = 3m + 1$ and substitute it in the equation (2.16) we ge:

$$\begin{aligned}
X[3m + 1] &= \sum_{n=0}^{N/3-1} x[n] \cdot W_N^{n(3m+1)} + \sum_{n=N/3}^{2N/3-1} x[n] \cdot W_N^{n(3m+1)} + \sum_{n=2N/3}^{N-1} x[n] \cdot W_N^{n(3m+1)} \\
&= \sum_{n=0}^{N/3-1} ((x[n] + x[n + N/3]W_3^1 + x[n + 2N/3]W_3^2) \cdot W_{N/3}^{nm}).
\end{aligned} \tag{2.18}$$

And finally, if we plug in $k = 3m + 2$ and substitute it in the equation (2.16) we get:

$$\begin{aligned}
X[3m + 2] &= \sum_{n=0}^{N/3-1} x[n] \cdot W_N^{n(3m+2)} + \sum_{n=N/3}^{2N/3-1} x[n] \cdot W_N^{n(3m+2)} + \sum_{n=2N/3}^{N-1} x[n] \cdot W_N^{n(3m+2)} \\
&= \sum_{n=0}^{N/3-1} ((x[n] + x[n + N/3]W_3^2 + x[n + 2N/3]W_3^1) \cdot W_{N/3}^{nm}).
\end{aligned} \tag{2.19}$$

This way, the DFT of length N can be calculated as three DFTs of length $N/3$. So the radix-3 FFT algorithm can be expressed as:

$$X[k] = \sum_{m=0}^{N/3-1} ((x[n] + x[n + N/3] + x[n + 2N/3]) \cdot W_{N/3}^{nm}) \tag{2.20}$$

$$X[k + N/3] = \sum_{m=0}^{N/3-1} ((x[n] + x[n + N/3]W_3^1 + x[n + 2N/3]W_3^2) \cdot W_{N/3}^{nm}) \tag{2.21}$$

$$X[k + 2N/3] = \sum_{m=0}^{N/3-1} ((x[n] + x[n + N/3]W_3^2 + x[n + 2N/3]W_3^1) \cdot W_{N/3}^{nm}) \tag{2.22}$$

The butterfly diagram for the radix-3 FFT algorithm can be represented as in Figure 2.4.

Similarly, if we change N_1 to 5, we get a radix-5 FFT algorithm. Then, coefficients would look like this:

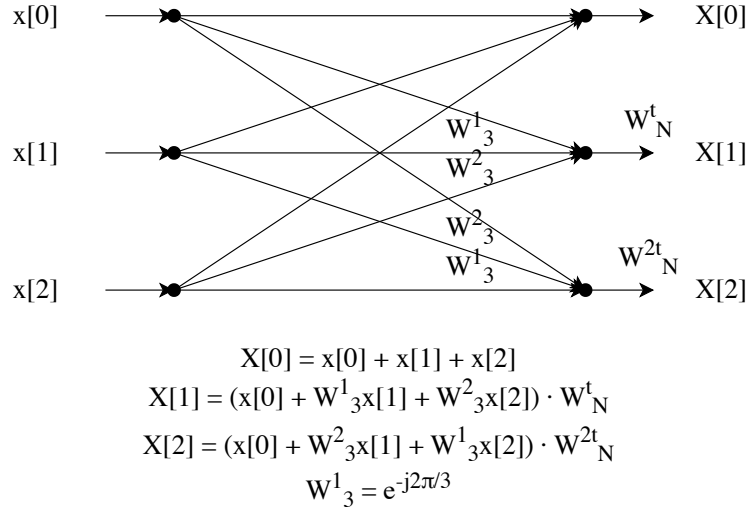


Figure 2.4: Graphical representation of the radix-3 FFT algorithm for $N = 8$.

$$\begin{aligned}
X[k] &= \sum_{n=0}^{N/5-1} ((x[n] + x[n + N/5] + x[n + 2N/5] \\
&\quad + x[n + 3N/5] + x[n + 4N/5]) \cdot W_{N/5}^{nm}) \quad (2.23)
\end{aligned}$$

$$\begin{aligned}
X[k + N/5] &= \sum_{n=0}^{N/5-1} ((x[n] + x[n + N/5]W_5^1 + x[n + 2N/5]W_5^2 \\
&\quad + x[n + 3N/5]W_5^3 + x[n + 4N/5]W_5^4) \cdot W_{N/5}^{nm}) \quad (2.24)
\end{aligned}$$

$$\begin{aligned}
X[k + 2N/5] &= \sum_{n=0}^{N/5-1} ((x[n] + x[n + N/5]W_5^2 + x[n + 2N/5]W_5^4 \\
&\quad + x[n + 3N/5]W_5^1 + x[n + 4N/5]W_5^3) \cdot W_{N/5}^{nm}) \quad (2.25)
\end{aligned}$$

$$\begin{aligned}
X[k + 3N/5] &= \sum_{n=0}^{N/5-1} ((x[n] + x[n + N/5]W_5^3 + x[n + 2N/5]W_5^1 \\
&\quad + x[n + 3N/5]W_5^4 + x[n + 4N/5]W_5^2) \cdot W_{N/5}^{nm}) \quad (2.26)
\end{aligned}$$

$$\begin{aligned}
X[k + 4N/5] &= \sum_{n=0}^{N/5-1} ((x[n] + x[n + N/5]W_5^4 + x[n + 2N/5]W_5^3 \\
&\quad + x[n + 3N/5]W_5^2 + x[n + 4N/5]W_5^1) \cdot W_{N/5}^{nm}) \quad (2.27)
\end{aligned}$$

The butterfly diagram for the radix-5 FFT algorithm can be represented

as in Figure 2.5.

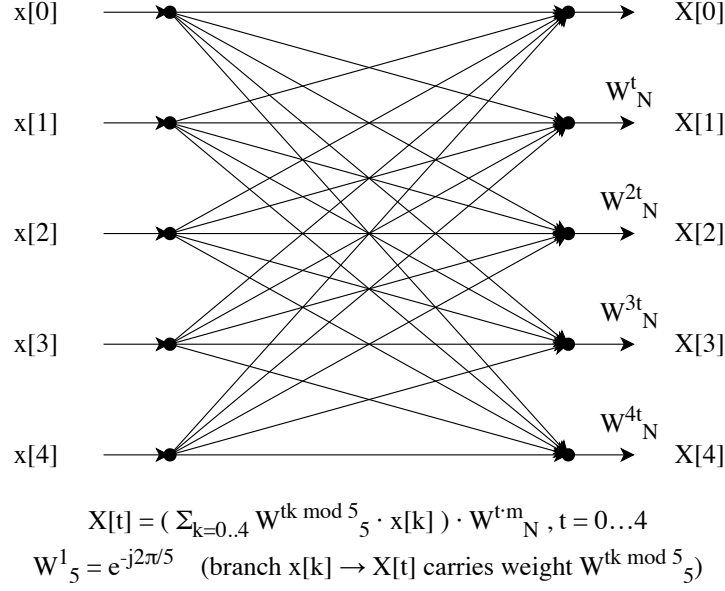


Figure 2.5: Graphical representation of the radix-5 FFT algorithm for $N = 8$.

2.2.3 Mixed-radix FFT

When a non-power-of-two FFT length is a request, the FFT algorithm can be implemented using a mixed-radix approach. This approach is a combination of prime-factor FFT algorithms (radix-2, radix-3, radix-5, etc.). The idea is to refactor the FFT length N into a product of prime factors $N = N_1 \times N_2 \times \dots \times N_k$ and then use the prime-factor FFT algorithms to compute the FFT. For example, if $N = 30$, we can factorize it into $N = 2 \times 3 \times 5$ and then use the radix-2, radix-3 and radix-5 FFT algorithms to compute the FFT.

The flowchart for the mixed-radix FFT algorithm on $N = 30$ can be constructed step by step as follows. Decompose the whole calculation into three stages: radix-2, radix-3 and radix-5. The first stage is the radix-2 FFT algorithm as represented in Figure 2.6.

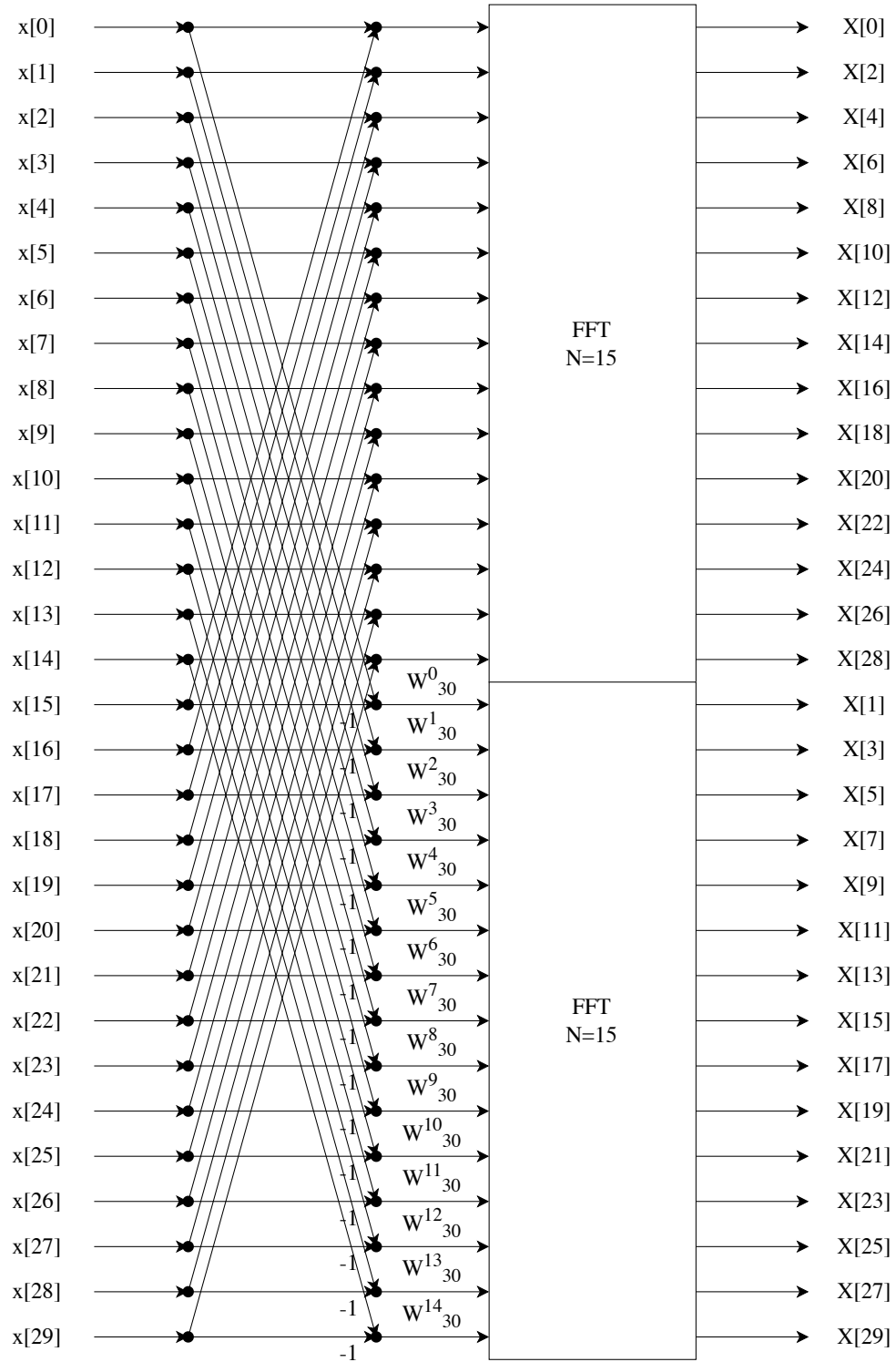


Figure 2.6: Flowchart for the mixed-radix FFT algorithm on $N = 30$.

Then the rest two FFT stages of length 15 can be decomposed into two stages: radix-3 and radix-5. The first stage is the radix-3 FFT algorithm as represented in Figure 2.7.

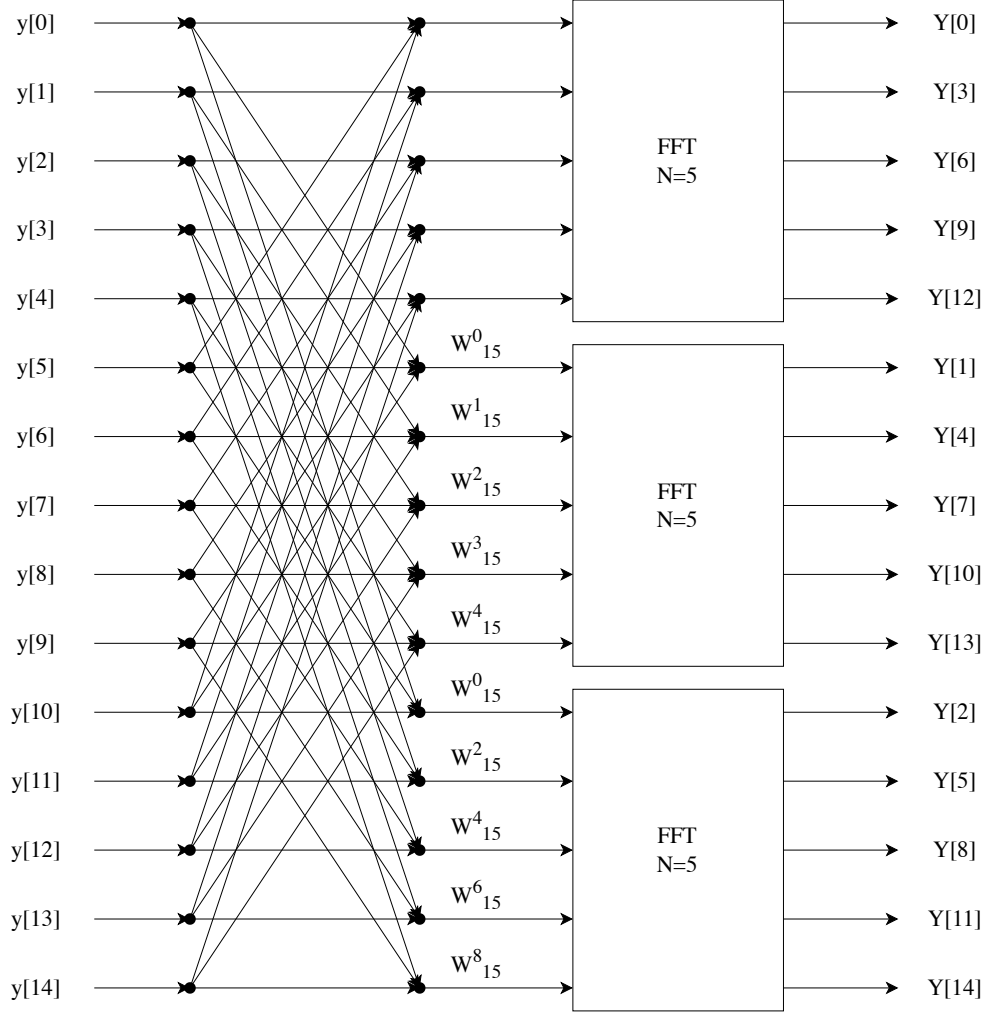


Figure 2.7: Flowchart for the mixed-radix FFT algorithm on $N = 30$.

The $N = 5$ FFT stages are just radix-5 FFT butterflies denoted in Figure 2.5.

So, the total flowchart for the mixed-radix FFT algorithm on $N = 30$ can be represented as in Figure 2.8.

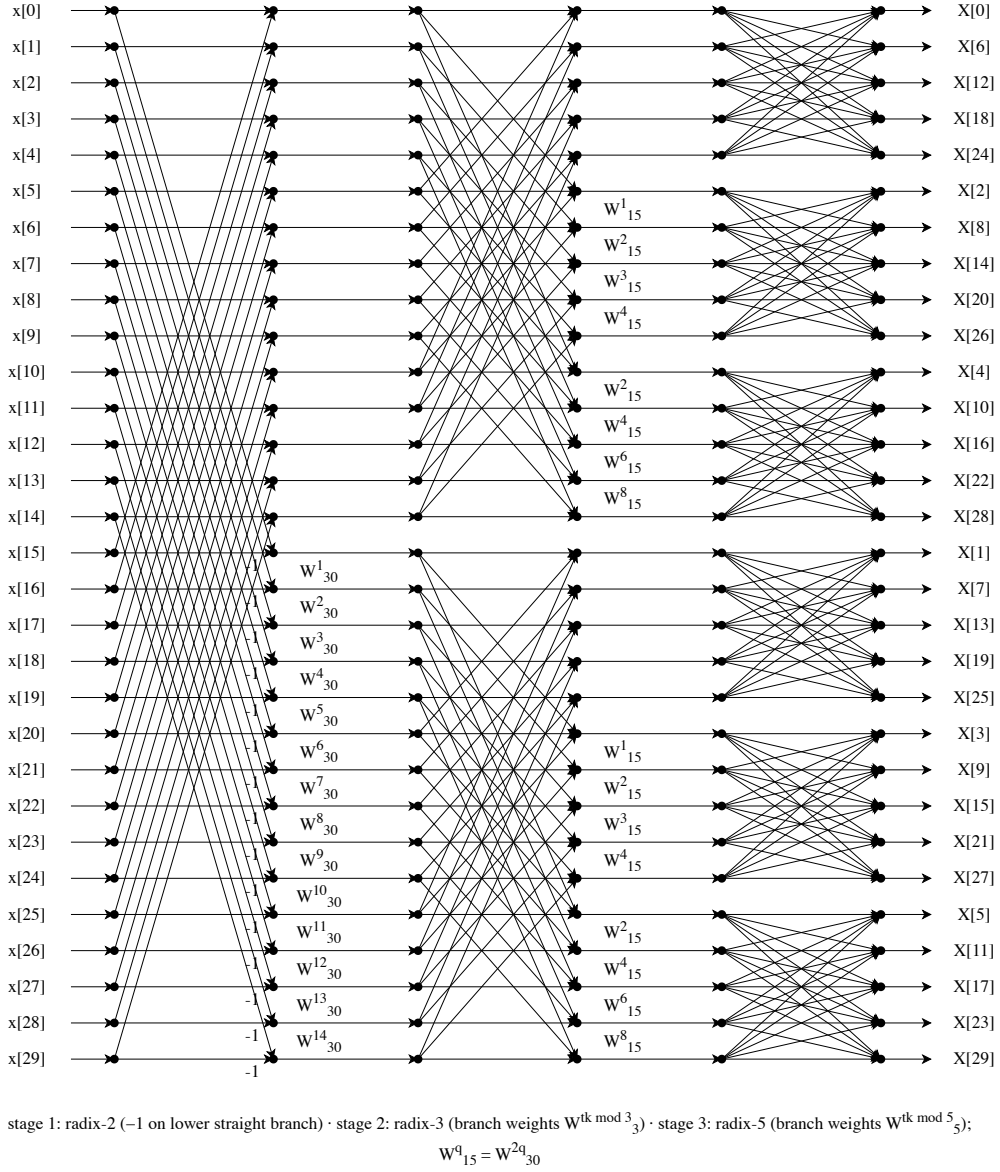


Figure 2.8: Flowchart for the mixed-radix FFT algorithm on $N = 30$.

2.2.4 Digit inversion

The DIF algorithm output samples come in scrambled order no matter the length of the FFT. To get samples in the correct order, a digit inversion is required. The digit inversion is a process of reversing the order of the digits of sample's index in the correct base number system.

Radix-2 FFT algorithm shows the simplest case of digit inversion. By

inspecting the indices of output samples in the full butterfly diagram for $N = 8$ (Figure 2.2) it can be seen that the indices are in the order of 0, 4, 2, 6, 1, 5, 3, 7. In the Table 2.5 the indices are presented in decimal and binary systems in the first two columns.

Index	Binary	Inverted Binary	Inverted Decimal
0	000	000	0
4	100	001	1
2	010	010	2
6	110	011	3
1	001	100	4
5	101	101	5
3	011	110	6
7	111	111	7

Table 2.5: Indices of output samples in the full butterfly diagram for $N = 8$.

Simply by reversing the order of the digits of the index in the binary system, the inverted index in the decimal system is obtained.

Similar approach can be applied to the radix-3 and radix-5 FFT algorithms. They would require digit inversion in the base 3 and base 5 number systems respectively.

A more complex case is the mixed-radix FFT algorithm. It requires digit inversion in the base 2, base 3 and base 5 number systems together. The index is written in a mixed-radix positional system instead of binary, and then the digits are reversed together with the base of each digit.

Concretely, any index $n \in [0, N)$ can be written uniquely as:

$$n = d_1 \cdot \frac{N}{r_1} + d_2 \cdot \frac{N}{r_1 r_2} + \cdots + d_{k-1} \cdot \frac{N}{r_1 r_2 \cdots r_{k-1}} + d_k \cdot 1, \quad d_i \in \{0, 1, \dots, r_i - 1\}, \quad (2.28)$$

where d_i are the digits of the index n in the mixed-radix positional system defined by the factorization $N = r_1 r_2 \cdots r_k$ into radices r_i . The digit-inverted index $\tilde{n}$ is then obtained by reversing the order of the digits, where the place values are now determined by the reversed sequence of radices $(r_k, r_{k-1}, \dots, r_1)$:

$$\tilde{n} = d_k \cdot \frac{N}{r_k} + d_{k-1} \cdot \frac{N}{r_k r_{k-1}} + \cdots + d_2 \cdot \frac{N}{r_k r_{k-1} \cdots r_2} + d_1 \cdot 1. \quad (2.29)$$

As an example, consider $N = 15 = 5 \cdot 3$, i.e. $r_1 = 5$ and $r_2 = 3$. According to (2.28), an index n is decomposed as $n = d_1 \cdot 3 + d_2$ with $d_1 \in \{0, 1, 2, 3, 4\}$

and $d_2 \in \{0, 1, 2\}$, and the digit-inverted index given by (2.29) is $\tilde{n} = d_2 \cdot 5 + d_1$. The resulting permutation of all fifteen indices is shown in Table 2.6.

Index	Digits (d_1, d_2)	Inverted digits (d_2, d_1)	Inverted index $\tilde{n}$
0	(0, 0)	(0, 0)	0
1	(0, 1)	(1, 0)	5
2	(0, 2)	(2, 0)	10
3	(1, 0)	(0, 1)	1
4	(1, 1)	(1, 1)	6
5	(1, 2)	(2, 1)	11
6	(2, 0)	(0, 2)	2
7	(2, 1)	(1, 2)	7
8	(2, 2)	(2, 2)	12
9	(3, 0)	(0, 3)	3
10	(3, 1)	(1, 3)	8
11	(3, 2)	(2, 3)	13
12	(4, 0)	(0, 4)	4
13	(4, 1)	(1, 4)	9
14	(4, 2)	(2, 4)	14

Table 2.6: Digit inversion for the mixed-radix decomposition $N = 15 = 5 \cdot 3$.

Note that the digits (d_1, d_2) are read in bases $(5, 3)$, while the inverted digits (d_2, d_1) are read in the reversed bases $(3, 5)$. The obtained permutation $(0, 5, 10, 1, 6, 11, 2, 7, 12, 3, 8, 13, 4, 9, 14)$ is exactly the order in which the output samples of the whole DIF FFT of length $N = 15$ appear: the first (radix-5) stage decimates the output indices into five groups according to $k \bmod 5$, i.e. $\{X[0], X[5], X[10]\}$, $\{X[1], X[6], X[11]\}$, and so on, while the final (radix-3) stage produces each group in natural order.

Digit inversion is important step in mixed-radix FFT algorithm. It ensures that the output samples are in the correct order.

2.3 Inverse FFT

Forward DFT was discussed in the previous section. Inverse DFT is the inverse operation of forward DFT. It is defined as:

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot W_N^{-nk} \quad (2.30)$$

where $W_N = e^{-j\frac{2\pi}{N}}$ is the twiddle factor.

The inverse DFT can be implemented using the same algorithm as the forward DFT in two ways:

- Swapping the input and output samples real and imaginary parts or
- Conjugating the input and output samples.

Both ways require division by N in the end of the algorithm.

Swapping and conjugating methods are represented in Figures 2.9a and 2.9b respectively.

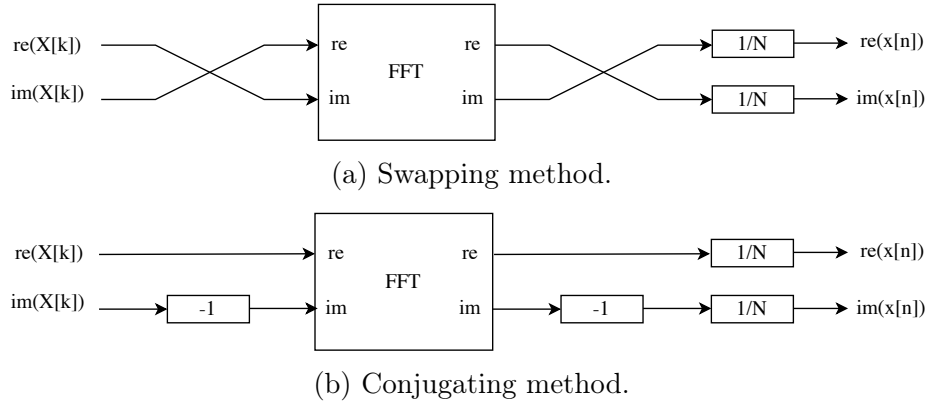


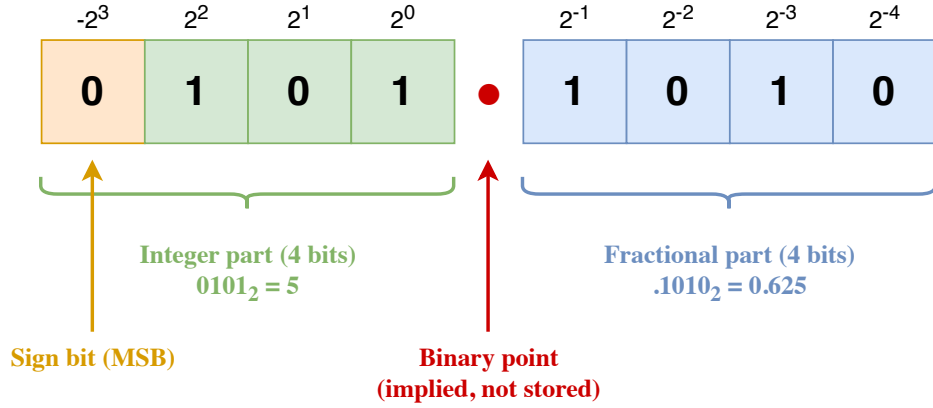
Figure 2.9: Inverse DFT calculation using the forward FFT algorithm.

The conjugate method requires more hardware resources than the swapping method because the negation of the imaginary part may need a full adder to be implemented if it's represented in fixed-point format (effectively two's complement). Swapping method requires only a simple bit swap.

2.4 Fixed-point number representation

Hardware implementations of numerical algorithms often have a requirement to be resource-efficient and as fast as possible while maintaining good enough mathematical performance (such as SNR or Signal-to-Noise Ratio). Floating-point arithmetic is not always the best choice due to its high compute latency and big resource footprint. To achieve this, so called fixed-point arithmetic is often used. It is a form of integer arithmetic that is used to represent real numbers in a fixed-precision format.

Fixed-point number representation can be illustrated as in Figure 2.10.



$$\text{Value} = 2^2 + 2^0 + 2^{-1} + 2^{-3} = 4 + 1 + 0.5 + 0.125 = 5.625$$

$$\text{Range: } [-8, +7.9375] \quad \text{Resolution: } 2^{-4} = 0.0625$$

Figure 2.10: Fixed-point representation — signed Q4.4, value = 5.625.

It consists of a fixed number of bits for the integer part and a fixed number of bits for the fractional part. Between integer and fractional part there is an implied binary point. There is also a sign bit on the MSB position. The value of the fixed-point number is then given by the formula:

$$value = \frac{integer}{2^{fractional}} \quad (2.31)$$

where *integer* is the integer part and *fractional* is the fractional part.

Fixed-point number representation is a form of integer arithmetic that is used to represent real numbers in a fixed-precision format.

The most interesting attribute of fixed-point number representation is that it allows for a very efficient implementation of arithmetic operations by using simple integer arithmetics. For example, addition of two fixed-point numbers is equivalent to addition of two integers. Multiplication of two fixed-point numbers is equivalent to multiplication of two integers, with the caveat that the result is shifted by the number of fractional bits to fit into the fixed-point format.

Handling of overflows and underflows is a bit more complex. If the result of an arithmetic operation is too large to fit into the fixed-point format, it will overflow. If the result is too small to fit into the fixed-point format, it will underflow. These events are catastrophic for the mathematical performance of the algorithm since they lead to strong spurs in the signal spectrum. A logical step would be to grow the fixed-point format (increase the number of bits for the integer and fractional parts) to accommodate the result of the operation.

This is fine if such a bit growth is allowed by the resource constraints. But, when such is not the case or not desirable, either saturation mechanism or scaling mechanism can be used. Saturation mechanism sets the result to the maximum or minimum value of the fixed-point format. Scaling mechanism scales the result by a factor that accomodates the gained number of bits by the operation to the detriment of mathematical preformance (some LSB bits are discarded thus SNR is degraded).

Chapter 3

Hardware Architecture

High performance FFT implementation is a must for the telecommunication systems. That requirement, with direct pipelined implementation of mixed-radix FFT algorithm in hardware would be a translation of the diagram in Figure 2.8 into hardware. Data would have to be buffered in between stages where each stage would have N inputs and N outputs and process N samples in each clock cycle.

This achieves maximum possible throughput of the FFT algorithm, and is implemented by having N complex adders and up to $N/2 - 1$ complex multipliers for samples that need to be multiplied by non-trivial twiddle factors.

The first problem with this approach is that it requires extremely large number of hardware resources to implement complex adders and multipliers. That large design is not feasible or even impossible to implement for FFT sizes that are required for precoding in 5G, displayed in Table 2.4. The second problem is that all samples are required at the same time. That requires enough memory to store N samples between each of the stages.

To efficiently implement the mixed-radix FFT algorithm in hardware, a more efficient architecture is needed. The resource inventory of modern FPGAs includes blocks like fast carry chains that implement fast adders, DSP blocks that implement pipelined multipliers and MAC units and on-chip fast SRAM blocks (e.g. BlockRAM in Xilinx FPGAs). Efficient use of these blocks is key to the success of the hardware implementation of any FPGA-based DSP algorithm, not just the mixed-radix FFT algorithm.

3.1 Streaming Mixed-Radix FFT Architecture

The streaming mixed-radix FFT architectures are pipelined implementations of the mixed-radix FFT algorithm. They are designed to solve the resource utilization problem by processing samples sequentially. Commonly, they process one sample in a clock cycle but there are also architectures that process multiple samples in a clock cycle. Streaming architectures reuse a single FFT butterfly per stage to process different samples sequentially.

The most common architectures are SDF (Single-path Delay Feedback) and MDF (Multi-path Delay Feedback) architectures. This master's thesis focuses on the SDF architecture.

3.1.1 SDF Architecture

Single-path Delay Feedback (SDF) architecture is one of the earliest implementations of the mixed-radix FFT algorithm and it appeared in [1]. It is a pipelined implementation of the mixed-radix FFT algorithm that processes samples sequentially by using a single full FFT butterfly per stage.

SDF FFT is composed of s stages that contain one input and one output each. Order of samples at the input of the i -th stage is the order of the input of the i -th stage in the signal flow graph of the mixed-radix FFT algorithm, top to bottom. The output of the i -th stage is the input of the $(i + 1)$ -th stage. The architecture is shown in Figure 3.1 for radix R FFT, where R is the radix of the FFT.

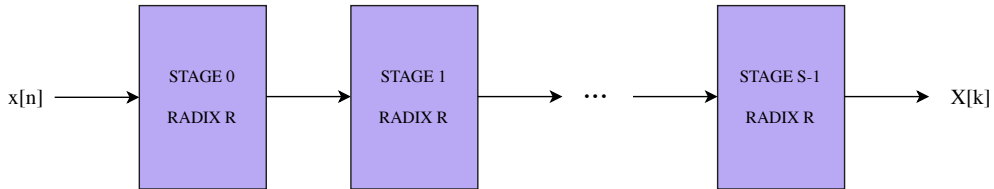


Figure 3.1: SDF architecture for radix R FFT.

One stage of the SDF FFT architecture is shown in Figure 3.2.

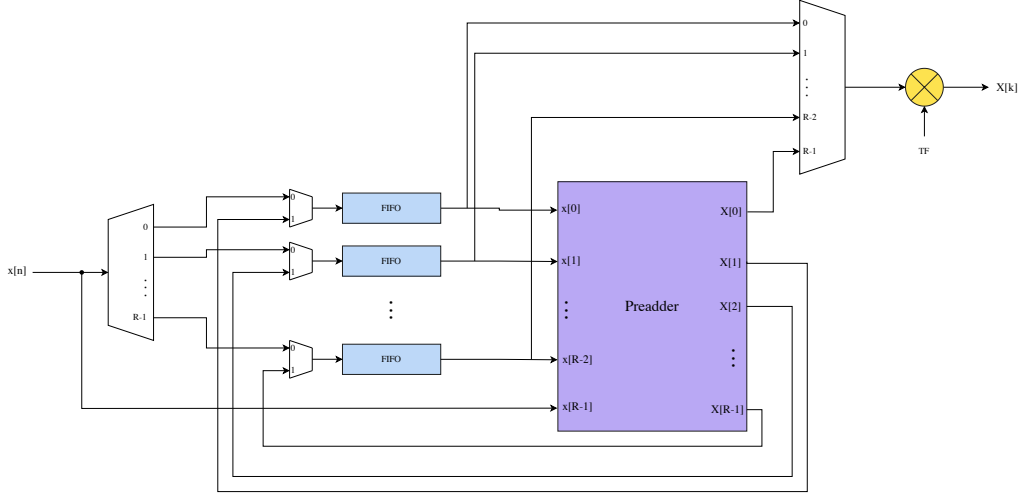


Figure 3.2: One stage of the SDF architecture for radix R FFT.

The data path of the stage consists of three main blocks: FIFOs, a Preadder and a Rotator. Preadder is a block that performs the butterfly computations and is literally a to-hardware translation of the butterfly diagrams from either radix-2, radix-3 or radix-5 FFT algorithms, or all of them combined in a reconfigurable manner, which will be discussed later. Rotator is a block that performs the twiddle factor multiplication and is a simple complex multiplier.

The idea of the SDF architecture is to have a single butterfly per stage and to process samples sequentially by using a single full FFT butterfly per stage. This requires serialization of operations in a single stage to be performed on a single butterfly and rotator. This is achieved by FIFOs that align correct samples to the input of the Preadder.

To illustrate the stream flow of data and the need for FIFOs, let's consider the SDF architecture for radix-2 FFT. The flow of data stream is shown in Figure 3.3.

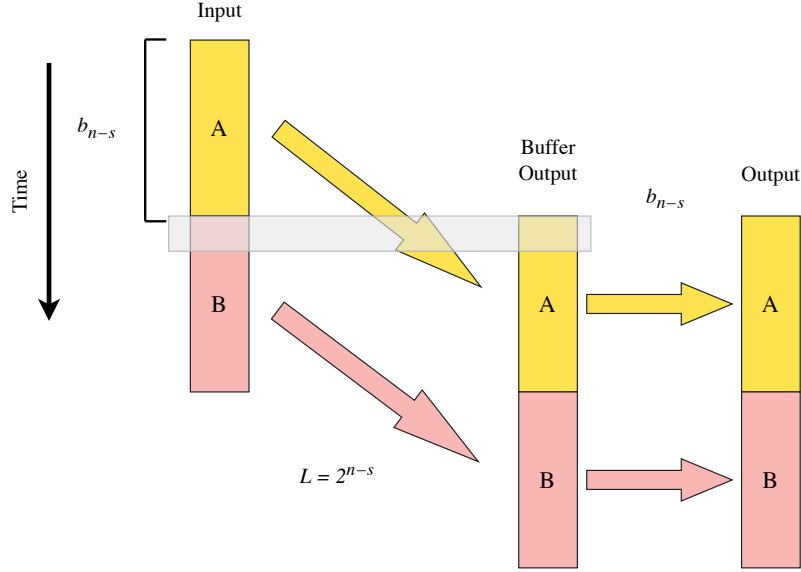


Figure 3.3: Data flow of the SDF architecture for radix-2 FFT.

One could see that to perform the computation on correct samples, a sample $x[n]$ needs to be aligned with the sample $x[n + N/2]$. This is achieved by the FIFO that stores the incoming first half of the samples and reads them when the second half of the samples starts arriving. This way all samples $N/2$ samples apart are put together into the Preadder for the butterfly computation.

The radix-2 butterfly (Figure 2.2) performs an addition and a subtraction of the input samples. With this architecture, the additions result of the Preadder can be passed to the Rotator right away and the subtraction result (generated in the same cycle as the addition result) can be feed back to the FIFO and passed to the Rotator when all addition results are finished generating. This is possible because the FIFO, full after the last sample of the first half of the samples are buffered, is being read out when each new sample of the second half of the samples arrives. These FIFO reads make space each cycle for a new sample to be buffered, and since we don't need input-sample buffering yet (we buffer only the first half of the samples), we can use the same FIFO for both input and output.

Radix-3 example of this dataflow is shown in Figure 3.4. Instead of buffering just the first half of the samples, it buffers the first two thirds of the samples in two separate buffers where each one buffers one of the thirds and reads them when the last third of the samples starts arriving. This way all samples $N/3$ samples apart are put together into the Preadder for the butterfly computation. The same would be true for a radix-5 stage.

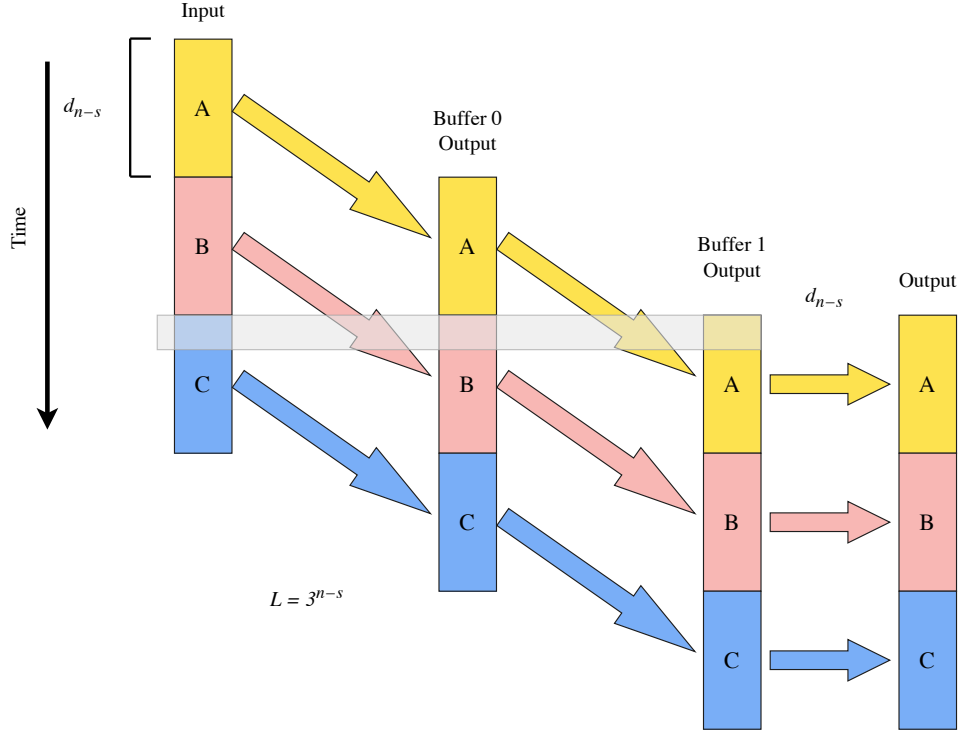


Figure 3.4: Data flow of the SDF architecture for radix-3 FFT.

The depth of the FIFOs is determined by the butterfly size. The butterfly size is the number of elementary butterfly operations in a stage. For example, for radix-2 $N = 2$ FFT, the butterfly size of its only stage is 1 so its FIFO depth is 1. For radix-3 $N = 9$ FFT, the butterfly size of its first stage (there are two stages in total) is 3 because it essentially performs 3 butterfly computations on 9 samples in total, so its FIFO depth is 3 for both of the FIFOs.

3.2 Microarchitecture

To fulfil the requirements of the 5G NR standard in the first place, all 53 FFT sizes need to be supported by the hardware implementation. In the table 3.1 the required FFT sizes factorized into $2^i \cdot 3^j \cdot 5^k$ format are presented.

Table 3.1: The 53 required FFT sizes factorized as $N = 2^i \cdot 3^j \cdot 5^k$.

N	i	j	k	stages	N	i	j	k	stages
12	2	1	0	3	864	5	3	0	8
24	3	1	0	4	900	2	2	2	6
36	2	2	0	4	960	6	1	1	8
48	4	1	0	5	972	2	5	0	7
60	2	1	1	4	1080	3	3	1	7
72	3	2	0	5	1152	7	2	0	9
96	5	1	0	6	1200	4	1	2	7
108	2	3	0	5	1296	4	4	0	8
120	3	1	1	5	1440	5	2	1	8
144	4	2	0	6	1500	2	1	3	6
180	2	2	1	5	1536	9	1	0	10
192	6	1	0	7	1620	2	4	1	7
216	3	3	0	6	1728	6	3	0	9
240	4	1	1	6	1800	3	2	2	7
288	5	2	0	7	1920	7	1	1	9
300	2	1	2	5	1944	3	5	0	8
324	2	4	0	6	2160	4	3	1	8
360	3	2	1	6	2304	8	2	0	10
384	7	1	0	8	2400	5	1	2	8
432	4	3	0	7	2592	5	4	0	9
480	5	1	1	7	2700	2	3	2	7
540	2	3	1	6	2880	6	2	1	9
576	6	2	0	8	2916	2	6	0	8
600	3	1	2	6	3000	3	1	3	7
648	3	4	0	7	3072	10	1	0	11
720	4	2	1	7	3240	3	4	1	8
768	8	1	0	9					

The number of radix-2, radix-3 and radix-5 stages to cover all FFT sizes is 10, 6 and 3 respectively. This means that the SDF architecture needs to be able to support 10 radix-2 stages, 6 radix-3 stages and 3 radix-5 stages but not all of them active at the same time. Implementing all of them at the same time would require a lot of hardware resources which is feasible, but not optimal. Therefore, the SDF architecture is implemented in a reconfigurable manner because not all of the stages are needed at the same time. In fact, maximum number of stages that are active in one configuration is 11 for the case of the $N = 3072$. So, if a stage can be reconfigured to be a radix-2 stage,

a radix-3 stage or a radix-5 stage, or some needed combination of them, the hardware implementation can be optimized to use only the necessary number of stages.

This reconfigurable architecture leads to a high level block diagram of the SDF architecture as shown in Figure 3.5 where each stage is a reconfigurable block that can be configured as a radix-2 stage, a radix-2/3 stage or a radix-2/3/5 stage. If a stage is not needed, it can be bypassed by reconfiguring it to be in bypass mode.

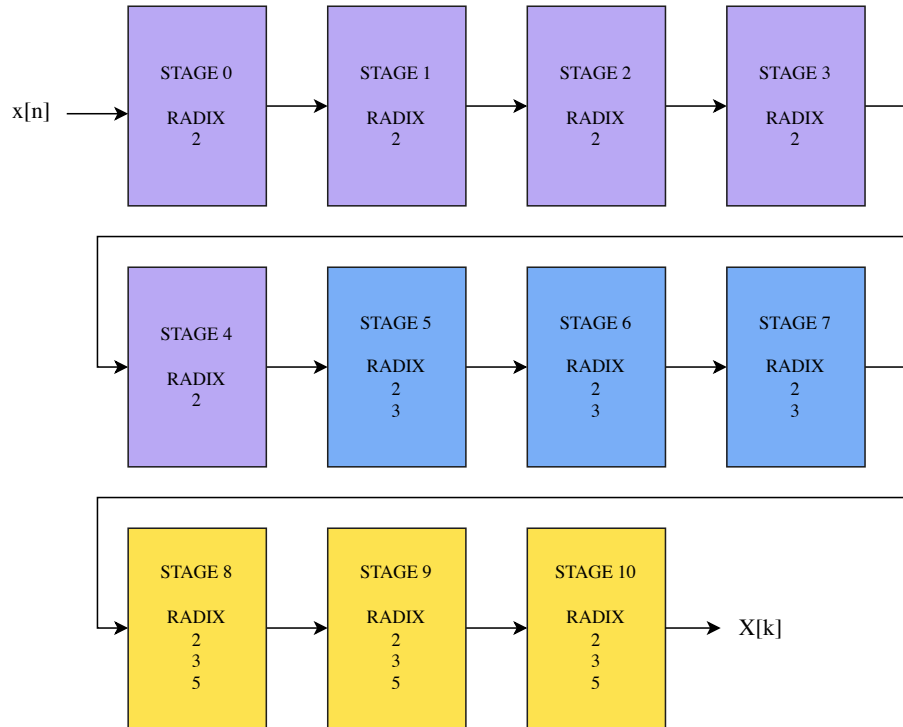


Figure 3.5: High level block diagram of the reconfigurable SDF architecture.

So, three levels of stage capabilities are needed:

- Radix-2 stage,
- Radix-2/3 stage,
- Radix-2/3/5 stage.

3.2.1 Radix-2 Stage

The proposed radix-2 stage is shown in Figure 3.6. Besides the FIFO, Preader and Rotator, which we already discussed, it contains control logic that is

responsible for the reconfiguration of the stage, a Twiddle Factor Unit and another FIFO at the output. The role of the reconfiguration in this stage is to bypass the stage. The Twiddle Factor Unit is responsible for the generation of the twiddle factors for the stage when a certain FFT size is selected. It will be discussed in more detail in the next section. All stages in the chain use ready/valid protocol for communication between them. Rotator has a free running pipeline inside it and a valid bit shift register to mark which sample is the valid one. So, the output Skid FIFO has a role of a skid buffer that can buffer valid samples of the Rotator when the next stage is not ready to receive the data, and stop the issue of new samples to Rotator. Another role of the Skid FIFO is to buffer the samples when the stage is bypassed and no samples are needed to be processed. This cuts combinational path for ready and valid signals between stages if the stage is bypassed.

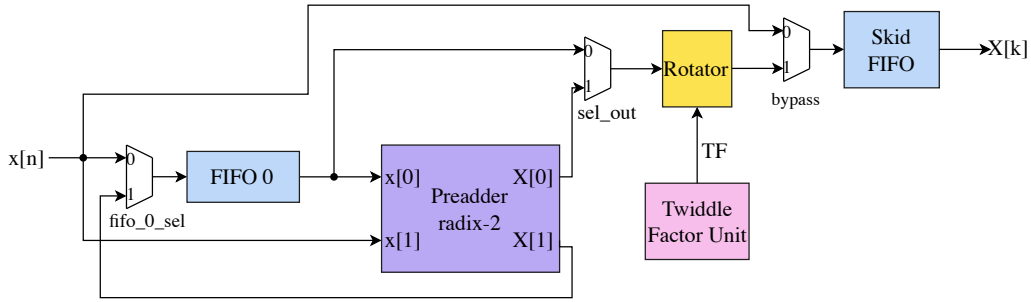


Figure 3.6: Reconfigurable radix-2 stage of the SDF architecture.

Preadder of a radix-2 stage is shown in Figure 3.7. It is a simple butterfly computation block that performs the addition and subtraction of the input samples and essentially has two signed adders. Each output passes through a $\gg 1$ rnd block, which halves the result and rounds it back to the memory word; this is the scaling step of the stage and is described in Section 3.2.6.

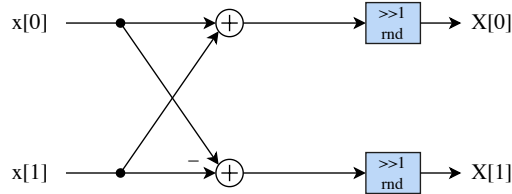


Figure 3.7: Preadder of the radix-2 stage.

3.2.2 Radix-2/3 Stage

The proposed radix-2/3 stage is shown in Figure 3.8. Besides the components also present in the radix-2 stage, it contains additional Result FIFOs at the Preadder outputs.

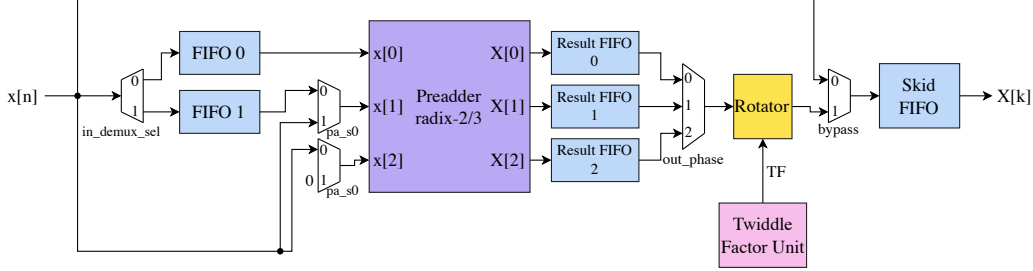


Figure 3.8: Reconfigurable radix-2/3 stage of the SDF architecture.

Radix-2/3 Preadder is shown in Figure 3.9. It is a more complex butterfly computation block that besides multiple additions and subtractions, it also performs two multiplications by fixed coefficients. One of the coefficients is either 1 or 0.5, depending on the stage radix configuration. Since both values are powers of two, this "multiplication" does not require a multiplier at all – it is implemented as a multiplexed arithmetic shift. The other coefficient is

$$k_6 = -\sin \frac{2\pi}{3} = \frac{j}{2} (W_3^{-1} - W_3^1) = -\frac{\sqrt{3}}{2} \approx -0.8660, \quad (3.1)$$

which is simply the imaginary part of the radix-3 twiddle factor $W_3 = -\frac{1}{2} - j\frac{\sqrt{3}}{2}$. Note that k_6 is a purely real number. A general complex multiplication $(a + jb)(c + jd)$ costs four real multiplications and two additions, but multiplying a complex sample by a purely real coefficient k reduces to $ka + jkb$, which is only two real multiplications and no additions. The remaining factor j that completes the radix-3 butterfly is pulled out of the coefficient and applied separately in the j block of Figure 3.9: multiplication by j is just a swap of the real and imaginary parts with one negation, so it costs a bit vector inversion and an adder in hardware (negation in two's complement is a bit inversion and addition of 1).

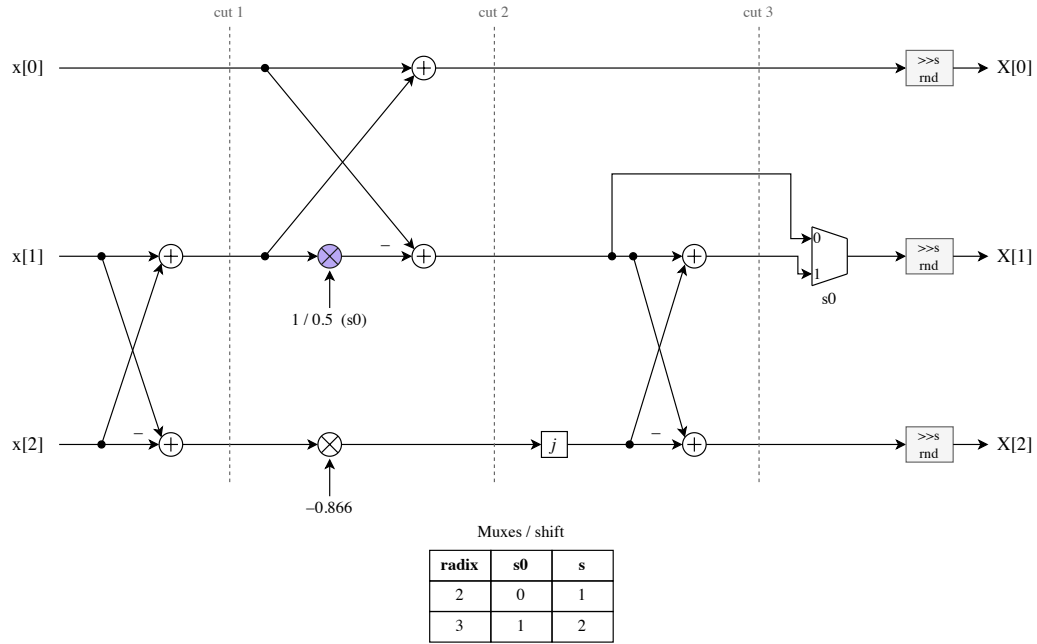


Figure 3.9: Radix-2/3 Preadder.

Dashed vertical lines indicate the pipeline stages inserted to cut long combinational paths through the preadder. This adds three clock cycles of latency to the stage. but it is a necessary to achieve the desired maximum clock frequency. The introduced latency creates a data hazard that's resolved by addition of the Result FIFOs. If the direct write-back to the input FIFOs is considered with a Preadder that has latency of one or more clock cycles, for example 2 clock cycles, the following hazard scenario would occur:

- Currently, the third third of samples is being processed by the Preadder.
- Last three samples for calculation are passed to Preadder inputs.
- Because of the latency of Preadder, the three samples are not available at the Preadder outputs in the same clock cycle so they cannot be written back to the input FIFOs.
- In the next clock cycle, the first sample of the new stream of samples is coming to the first input FIFO and the FIFO is being written with new samples even though all samples from the previous stream are not yet written back to it.
- After two clock cycles, the three samples are available at the Preadder outputs and can be written back to the input FIFOs.

- The new stream of samples is overlapped with the previous stream and the samples are lost.

This would cause a data hazard because some samples are lost. This is resolved by either gating the new stream of samples, which creates bubbles, or by the addition of new set of FIFOs (with the depth equal to the depth of the input FIFOs + the pipeline depth of the Preadder to accomodate the latency like Skid FIFO does for the Rotator output) at the Preadder outputs to accept the Preadder results. Result FIFOs are then read and samples are passed to Rotator pipeline for twiddle factor multiplication. At the cost of doubling the memory requirements of a stage, the second option is chosen.

Table below the Preadder diagram shows the multiplexer select values for each of radix configurations, together with the scaling shift s applied in the $\gg s$ rnd blocks at the outputs: one bit in the radix-2 mode and two bits in the radix-3 mode (Section 3.2.6).

3.2.3 Radix-2/3/5 Stage

The proposed radix-2/3/5 stage is shown in Figure 3.10.

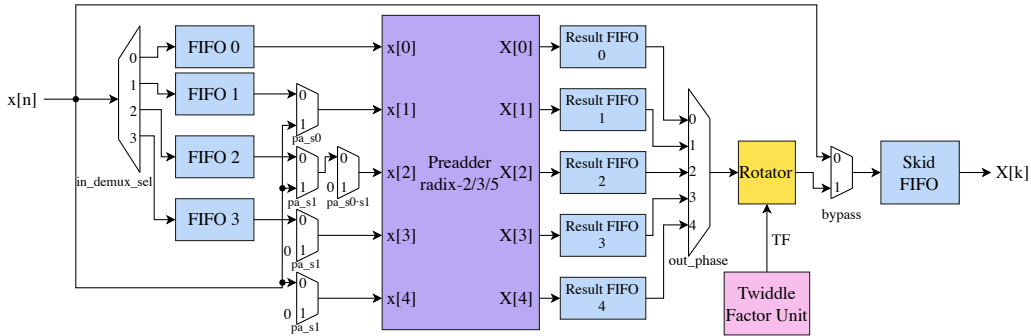


Figure 3.10: Reconfigurable radix-2/3/5 stage of the SDF architecture.

Radix-2/3/5 Preadder is shown in Figure 3.11. It extends the radix-2/3 structure with the adder levels and coefficient multipliers required by the radix-5 butterfly. The multiplexed shift coefficient now takes the values 1, 0.5 or 0.25 (again implemented as a shift), the shared multiplier selects between k_6 for the radix-3 mode and k_2 for the radix-5 mode, and three additional fixed multipliers apply the coefficients k_3 , k_4 and k_5 . Expressed both trigonometrically and through the twiddle factors W_5^k , the coefficients are:

$$k_2 = \frac{1}{2} \left(\cos \frac{2\pi}{5} - \cos \frac{4\pi}{5} \right) = \frac{1}{4} (W_5^1 + W_5^{-1} - W_5^2 - W_5^{-2}) = \frac{\sqrt{5}}{4} \approx 0.5590, \quad (3.2)$$

$$k_3 = j \left(\sin \frac{4\pi}{5} - \sin \frac{2\pi}{5} \right) = \frac{1}{2} (W_5^1 - W_5^{-1} - W_5^2 + W_5^{-2}) \approx -j 0.3633, \quad (3.3)$$

$$k_4 = -j \sin \frac{4\pi}{5} = \frac{1}{2} (W_5^2 - W_5^{-2}) \approx -j 0.5878, \quad (3.4)$$

$$k_5 = j \left(\sin \frac{2\pi}{5} + \sin \frac{4\pi}{5} \right) = \frac{1}{2} (W_5^{-1} - W_5^1 + W_5^{-2} - W_5^2) \approx j 1.5388. \quad (3.5)$$

These coefficients arise from the symmetries of the radix-5 twiddle factors: the real parts of W_5^k and W_5^{-k} are equal while their imaginary parts are opposite, which allows the DFT matrix to be factored into sums and differences of the inputs multiplied by the purely real combination k_2 and the purely imaginary combinations k_3 , k_4 and k_5 .

Note that every coefficient is again either purely real (k_2 , k_6) or purely imaginary (k_3 , k_4 , k_5), so by the argument given for the radix-2/3 preadder each of the four coefficient multipliers costs only two real multiplications instead of four, mapping to eight real multipliers (DSP blocks) instead of sixteen. For k_3 , k_4 and k_5 the zero real part is simply pruned by synthesis, while on the shared k_6/k_2 multiplier the factor j needed in the radix-3 mode is applied structurally by the multiplexed j block.

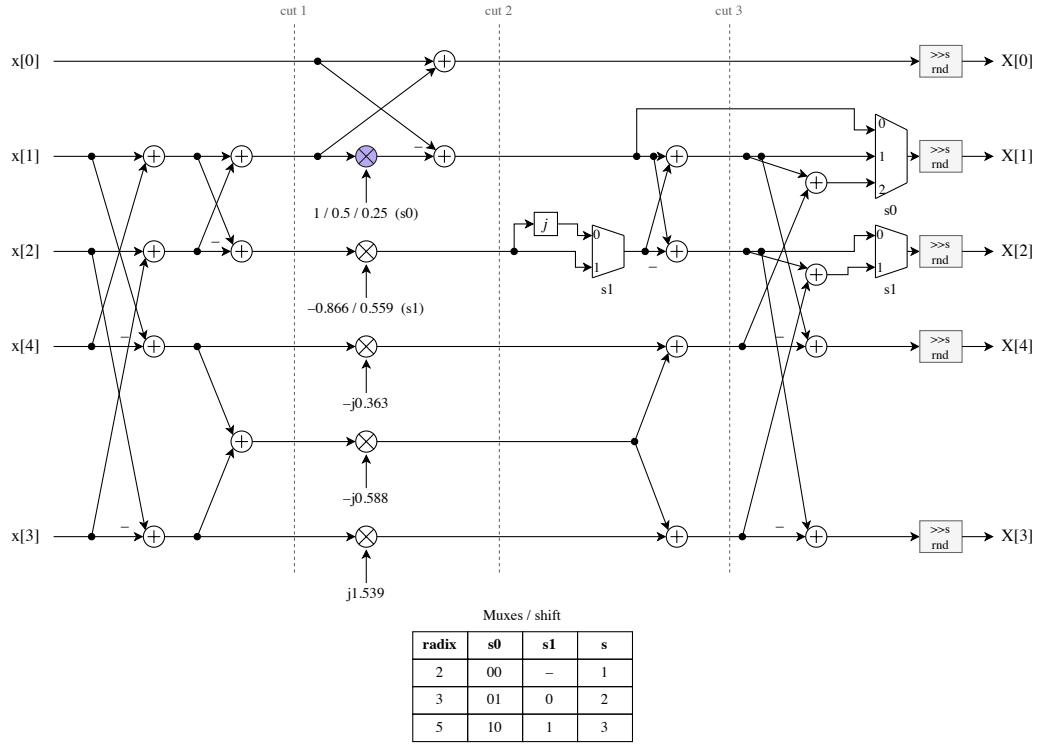


Figure 3.11: Radix-2/3/5 Preadder.

Dashed vertical lines indicate the pipeline stages inserted to cut long combinational paths through the preadder. This adds three clock cycles of latency to the stage just like in the radix-2/3 case.

Table below the Preadder diagram shows the multiplexer select values for each of radix configurations and the scaling shift s of the output $\gg s$ rnd blocks: one, two or three bits for the radix-2, radix-3 and radix-5 modes. The shift amount is selected by the same s0 signal that selects the radix mode.

3.2.4 Twiddle Factor Unit

The twiddle factor unit generates one twiddle factor for every output beat of the stage. Its block diagram is shown in Figure 3.12.

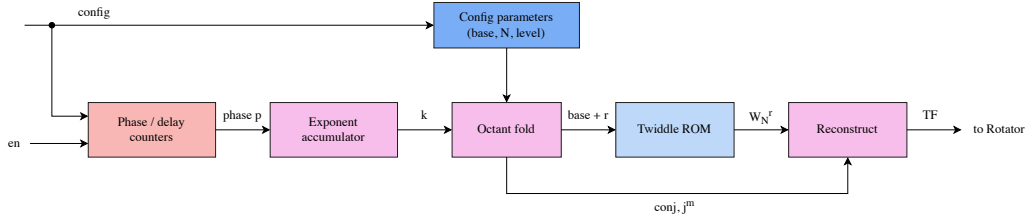


Figure 3.12: Twiddle factor unit.

The phase and delay counters (discussed in subsection 3.2.8) track the output arm p (from which "arm" of the Padder Result FIFOs the sample comes from) and the position n inside the block (index of the sample inside the arm block). From them the twiddle exponent $k = p \cdot n$ is formed by the exponent accumulator: it is cleared at the start of every block and incremented by p on every beat, so no multiplier is needed.

The twiddle factors are read from a ROM. Storing all N points of the unit circle would be wasteful, so only the first octant ($0 \leq r \leq N/8$) is stored and the symmetry of the unit circle is used for the rest. The octant fold block reduces the exponent k to the stored index r in three steps (half, quarter and octant of the circle); each step sets a flag: a conjugate flag and a two-bit code m of the unit multiplier j^m . The tables for all FFT sizes supported by the stage are packed into a single ROM, and the configuration parameters block supplies the base address and size N of the selected configuration. The ROM is read synchronously, so it maps to block RAM.

The flags travel alongside the ROM and are applied in the reconstruct block: the conjugate flag negates the imaginary part and the multiplication by j^m is a swap and negation of the components, so the reconstruction is done only with multiplexers. All blocks of the unit are registered and the stage delays the data samples by the same number of cycles, so each twiddle factor meets its sample at the rotator input.

3.2.5 Rotator

The rotator multiplies every output sample of the stage by the twiddle factor supplied by the twiddle factor unit. Its structure is shown in Figure 3.13.

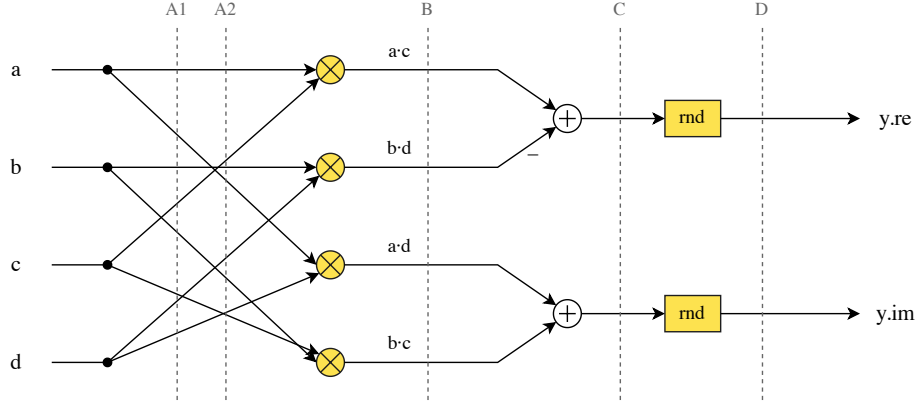


Figure 3.13: Rotator.

Unlike the preadder coefficients, the twiddle factor is a general complex number, so the full complex product is needed. With the sample $x = a + jb$ and the twiddle factor $W = c + jd$, the result is formed as $y = (ac - bd) + j(ad + bc)$, which takes four real multiplications and two additions. The products and their sums are kept in full precision; the result is quantized back to the memory word only once, in the rnd blocks, with half-up rounding.

The rotator is pipelined with five register cuts, marked with dashed lines in Figure 3.13: two operand registers (A1, A2), the product registers (B), the sum registers (C) and the rounding registers (D). The cuts are placed so that synthesis absorbs A1/A2, B and C into the input, multiplier and output registers of the four DSP blocks. A valid flag is delayed through the same number of registers, so the datapath runs freely without enables.

3.2.6 Scaling and Rounding

All samples stored or streamed in the design – in the delay, result and skid FIFOs and on the links between the stages – share one fixed-point memory word. A butterfly, however, grows the signal: a radix- r butterfly sums r inputs, so its outputs can be up to r times larger than the inputs. Without a countermeasure the word would overflow after a few stages. Growing the word by $\log_2 r$ bits in every stage would avoid the overflow, but it would make every FIFO, adder and multiplier downstream wider, and the FIFOs dominate the memory of the design.

Instead, the stages apply a fixed scaling schedule: every stage divides its butterfly outputs by a power of two that covers the growth of its radix, i.e. by 2^s with $s = \lceil \log_2 r \rceil$ – one bit for radix-2, two bits for radix-3 and three bits for radix-5. A power of two is chosen deliberately, because dividing by

three or five exactly would need a multiplier per output, while a division by 2^s is an arithmetic right shift, which costs no logic at all. The shift amount is selected by the same radix mode signal $s0$ that configures the preadder. Since the shift always covers the growth, the stage output fits back into the memory word by construction and no saturation logic is needed. The price is the one already discussed in Section 2.4: the bits shifted out are lost, so a small amount of quantization noise is added in every stage.

To keep this noise as low as possible the arithmetic inside the preadder is done in a wider word and without intermediate rounding: the adder tree, the algorithmic $1/2$ and $1/4$ shifts of the radix-3 and radix-5 butterflies and the sums of the coefficient products are computed exactly, and only the coefficient products themselves are trimmed once on the post-adder of the DSP block. The scaling shift and the return to the memory word are then fused into a single operation at the exit of the preadder, the $\gg s \text{ rnd}$ blocks in Figures 3.7, 3.9 and 3.11: half of an output LSB is added, the word is shifted right by s bits and truncated, which gives half-up rounding for free. In this way each stage rounds its data exactly once in the butterfly. The rotator adds a second rounding per stage, but no scaling: the magnitude of a twiddle factor is at most one, so the product needs no extra headroom and is only rounded back to the memory word, as described above.

Because the shift is $\lceil \log_2 r \rceil$ rather than $\log_2 r$, the radix-3 and radix-5 stages divide by four and eight instead of by three and five. After n_2 radix-2, n_3 radix-3 and n_5 radix-5 stages the chain therefore delivers

$$X'_k = X_k \cdot 2^{-S}, \quad S = n_2 + 2n_3 + 3n_5, \quad (3.6)$$

instead of the X_k/N convention. The residual gain between the two is corrected by the final scaler at the end of the chain (Figure 3.17), which multiplies every output by

$$c = \frac{2^S}{N} = \left(\frac{4}{3}\right)^{n_3} \left(\frac{8}{5}\right)^{n_5}, \quad (3.7)$$

a real constant between $4/3$ and about 5.7 for the supported sizes. The constant is computed for every configuration at elaboration time and selected together with the configuration at commit through the SCALE register. Correcting the gain once at the end of the chain is much cheaper than dividing exactly in every stage: since c is real, the scaler needs only one real multiplication per component (one DSP block each), the half-up rounding constant again rides on the post-adder, and the product is rounded once to the memory word. The scaler is a three-stage pipeline with the same credit-gated

output skid as the stage, so the output of the core is delivered as X_k/N with a registered handshake.

3.2.7 FIFOs

All buffers of the stage – the delay FIFOs, the result FIFOs and the output skid – are instances of one show-ahead FIFO. The read port always presents the oldest word together with a valid flag, and the read enable pops it; the consuming side can therefore inspect the word first and decide cycle by cycle whether to take it, which is exactly what the drain gate and the output handshake need.

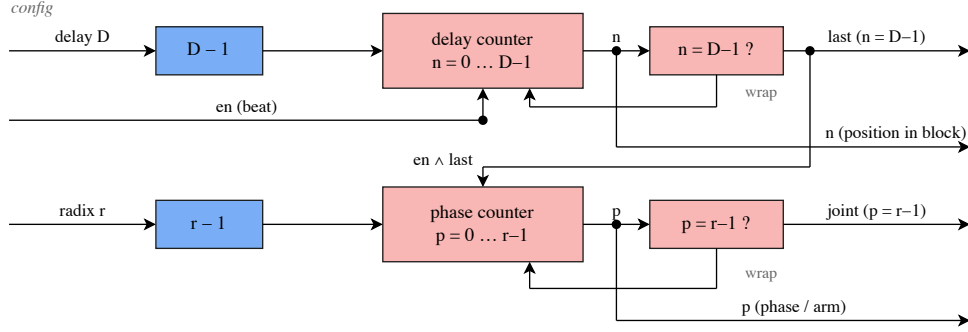
The storage is a simple dual-port block RAM with one write port and one read port. Two ports are required because in streaming operation the FIFO is written and read in the same cycle: while the joint phase pops one sample per delay FIFO, the fill phases of the next block are already writing new samples into them. The synchronous read of the block RAM has a latency of two cycles (memory read plus the RAM output register), so two additional fabric registers are placed in front of the read port. They hold the two oldest words, are refilled from the RAM in the background, and make back-to-back pops run at one word per cycle without bubbles; when the RAM is empty, writes bypass directly into these registers, so the FIFO also works as a delay line of depth one.

The reconfigurability of the depth costs nothing in the FIFO itself. Each FIFO is built at elaboration time for the largest configuration of its slot (depth N_{max}/r for the delay FIFOs), and the effective depth at runtime is set purely by occupancy: for a smaller FFT size the control simply writes fewer words (N/r for the selected configuration) before the joint phase starts popping them, so the same RAM acts as a shorter delay line. Only the delay compare value in the counters changes between configurations; the read and write pointers wrap at the physical depth, which does not need to be a power of two.

3.2.8 Phase and Delay Counters

All timing inside a stage is derived from one pair of cascaded counters, shown in Figure 3.14. The delay counter n advances on every beat and counts the position of the sample inside the current block, $0 \dots D - 1$, where $D = N/r$ is the delay of the stage for the selected configuration. When it wraps, the phase counter p advances by one; p counts the block, i.e. the arm of the butterfly the sample belongs to, $0 \dots r - 1$, and wraps at the radix. The beat

is therefore mapped to the pair (p, n) , as illustrated by the example at the bottom of the figure.



example, $r = 3, D = 4$:

beat	0	1	2	3	4	5	6	7	8	9	10	11	
n	0	1	2	3	0	1	2	3	0	1	2	3	
p	0	0	0	0	1	1	1	1	2	2	2	2	...

Figure 3.14: Phase and delay counters.

The two compare values $D - 1$ and $r - 1$ are the only inputs that depend on the configuration. They are registered once from the quasi-static configuration, so the per-beat wrap compares start from flip-flops and no subtractor sits on the counting path. The comparator outputs are exported as flags: last marks the final sample of a block and joint marks the final phase, in which the preadder is fed and the result FIFOs are written.

Every stage contains two instances of this counter pair, one enabled by the input beats and one by the output beats. The input pair steers the input demultiplexer and the FIFO write enables and defines the joint phase; the output pair steers the output multiplexer and supplies p and n to the twiddle factor unit, which forms the exponent $k = p \cdot n$ from them. Because the two pairs run independently, the input and output sides of the stage are decoupled, as described in Section 3.2.10.

3.2.9 Control Logic

The control logic contains no state machine: all control is derived from the phase and delay counters and the quasi-static configuration. Its structure is shown in Figure 3.15.

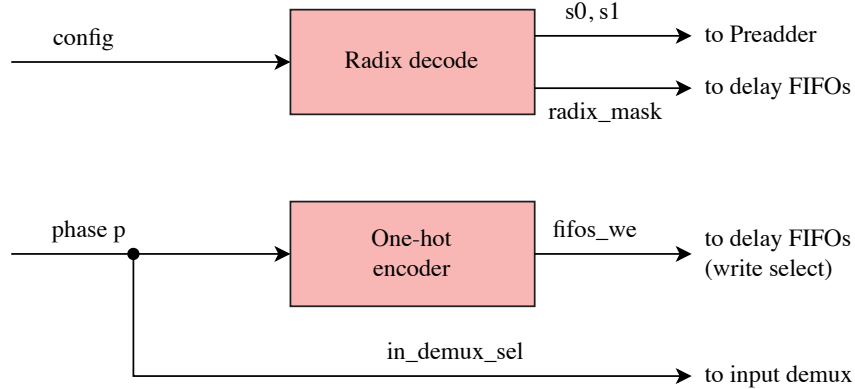


Figure 3.15: Stage control logic.

The radix decode block translates the configured radix into the preadder select signals s_0 and s_1 (with the values given in the preadder figures) and into radix_mask , which marks the delay FIFOs used by the selected radix; the FIFOs outside the mask are held inactive. The decode is registered, which is allowed because the configuration only changes between frames.

The input phase p directly drives the select of the input demultiplexer, and the one-hot encoder turns it into the write enables of the delay FIFOs, so every incoming sample is written into the FIFO of its arm. The write enables are qualified by the stage with the input handshake, so no gating is needed inside the control logic.

3.2.10 Flow Control

The stages of the chain are connected with a ready/valid handshake on both faces. Figure 3.16 shows the data and control flow of one stage: the datapath in the top row and the control that steers it below, with the dashed line marking the border between the input flow and the output flow.

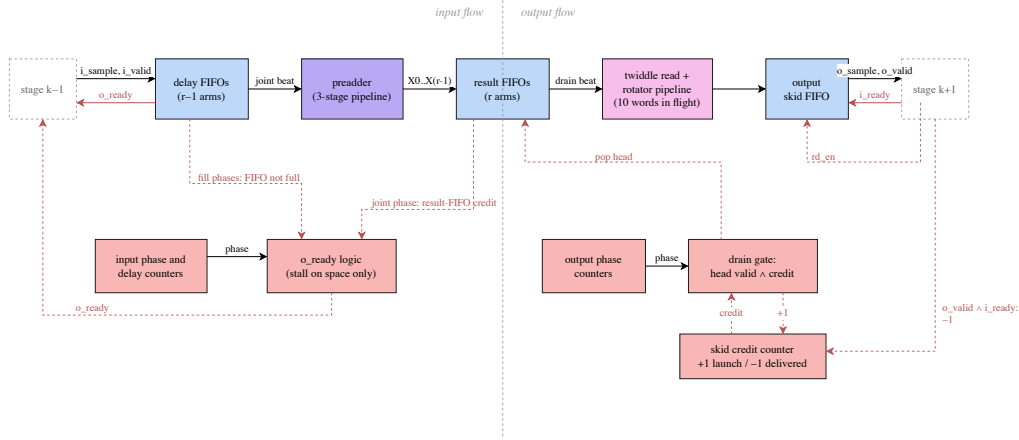


Figure 3.16: Data and control flow of the stage.

The two flows are fully decoupled and meet only in the FIFOs. The input side runs on its own phase and delay counters, and its `o_ready` logic stalls only on space: during the fill phases on the full flags of the delay FIFOs, and during the joint phase on the credit of the result FIFOs. The downstream `i_ready` never enters this logic, so the stage keeps accepting samples and prefills the delay FIFOs for the next block while the results of the previous block are still being read out of the result FIFOs – or are still waiting there because the receiving side is not ready yet.

The output side drains the result FIFOs on its own counters. A drain gate releases a result only when its FIFO head is valid and the skid credit counter shows room, because a released word then spends ten cycles in the twiddle read and the rotator pipeline and cannot be stopped anymore. If `i_ready` drops in the meantime, the word still lands safely in the output skid FIFO, whose depth covers all words in flight plus margin. The credit counter is incremented on every drain beat and decremented on every word delivered downstream (`o_valid` and `i_ready`), so the skid can never overflow.

With this partitioning there is no combinational path from `i_ready` to `o_ready` or from `i_valid` to `o_valid`: `i_ready` only pops the skid FIFO, and `o_ready` is derived from registered FIFO state. The stages therefore chain without timing loops, and a stall on any face is absorbed locally by the FIFOs instead of rippling through the whole chain within one cycle.

3.2.11 Top-level control

The chain of stages needs a top-level control logic to coordinate the reconfiguration of the stages and data flow between the stages. It is shown in Figure 3.17.

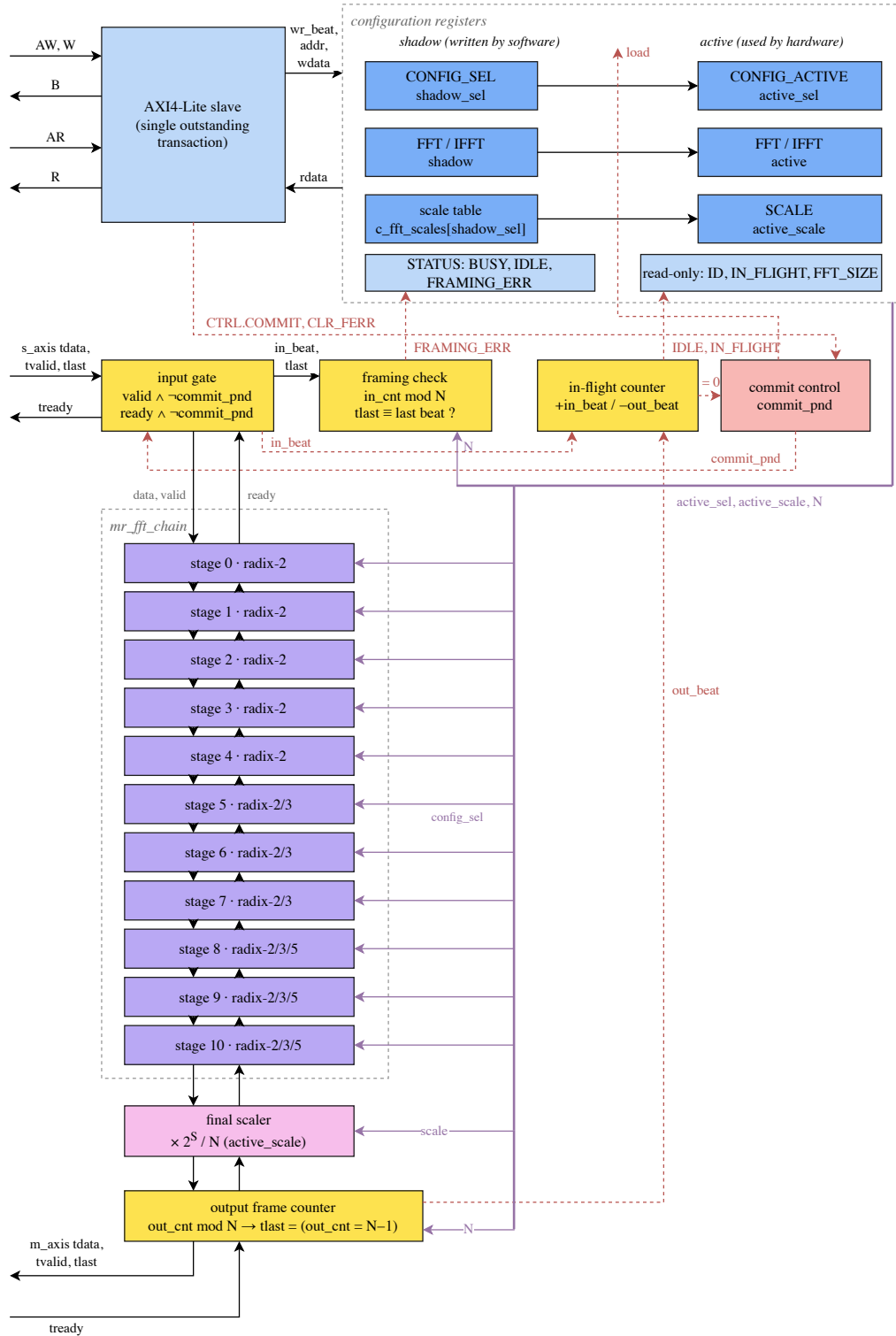


Figure 3.17: Top level: AXI4-Lite configuration interface, shadow and active registers, chain of stages and AXI4-Stream ports.

The core is configured over an AXI4-Lite slave with a small register map. The slave accepts the write address and data channels in the same beat and keeps a single transaction outstanding, which is sufficient for a quasi-static configuration. Every configurable value exists twice: software writes the shadow registers (CONFIG_SEL, the FFT/IFFT flag and, derived from CONFIG_SEL, the scaler constant), while the hardware only ever sees the active registers. The two are separated so that a new configuration can be written at any time without disturbing the frame that is currently in the chain.

The transfer from shadow to active is done by the commit control. A write of CTRL.COMMIT sets commit_pnd, which closes the input gate: the input tvalid is masked and tready is held low, so no new samples enter. The in-flight counter, incremented on every accepted input beat and decremented on every delivered output beat, tells when the chain is empty; at that moment the active registers are loaded from the shadow registers in one cycle and the gate reopens. Because the stages carry no state between frames once drained, no reset is needed. STATUS exposes BUSY (commit pending) and IDLE (in-flight counter at zero) so the software can poll the switch, or simply keep streaming while the gate handles the boundary.

The stages inside the chain are connected only through their ready/valid handshakes, and the active configuration select is distributed to all of them. A stage whose slot is not needed by the active size bypasses itself through its output skid, so the stream is re-registered in every slot and there are no chain-level multiplexers. The final scaler multiplies the chain output by the residual gain $2^S/N$ of the active size (Section 3.2.6), so that the output is delivered in the X/N convention.

The AXI4-Stream tlast marker is not carried through the chain. The delay FIFOs are set up for the active N , so an input tlast could not steer the datapath anyway; it is only checked by the input frame counter, and a tlast that does not fall on the last beat of a frame sets the sticky FRAMING_ERR bit in STATUS. On the output side tlast is regenerated from the output frame counter, which counts modulo the active N . This is exact because a commit only lands when the chain is drained, i.e. on a frame boundary, and both counters are cleared at that point.

Chapter 4

Software Reference Model

To implement the mixed-radix FFT algorithm in hardware, a software reference model must be developed. It is a software implementation of the mixed-radix FFT algorithm that is used for multiple purposes:

- Debugging of the hardware implementation,
- Verification of the hardware implementation,
- Model performance evaluation and optimization,
- Fast model design iterations.

The model is implemented in Python programming language for its simplicity, readability and ease of modification. It models all blocks of the mixed-radix FFT architecture described in this thesis - both control and data paths. Samples are represented as complex numbers in fixed-point format.

Bibliography

- [1] H. Groginsky and G. Works, “A pipeline fast Fourier transform,” IEEE Transactions on Computers, vol. C-19, no. 11, pp. 1015–1019, 1970.
- [2] ARM, *AMBA 4 AXI4-Stream protocol*, <https://developer.arm.com/documentation/ih0051/a/>.
- [3] 3GPP, *TS 38.211: NR; Physical channels and modulation*, V18.2.0, 3rd Generation Partnership Project, 2024.
- [4] 3GPP, *TS 38.104: NR; Base Station (BS) radio transmission and reception*, V18.5.0, 3rd Generation Partnership Project, 2024.

List of Figures

2.1	Graphical representation of the equations (2.12) and (2.13) for $N = 8$	12
2.2	Graphical representation of the equations (2.12) and (2.13) for $N = 8$	13
2.3	Graphical representation of the butterfly for $N = 8$	13
2.4	Graphical representation of the radix-3 FFT algorithm for $N = 8$	15
2.5	Graphical representation of the radix-5 FFT algorithm for $N = 8$	16
2.6	Flowchart for the mixed-radix FFT algorithm on $N = 30$. . .	17
2.7	Flowchart for the mixed-radix FFT algorithm on $N = 30$. . .	18
2.8	Flowchart for the mixed-radix FFT algorithm on $N = 30$. . .	19
2.9	Inverse DFT calculation using the forward FFT algorithm. . .	22
2.10	Fixed-point representation — signed Q4.4, value = 5.625. . .	23
3.1	SDF architecture for radix R FFT.	26
3.2	One stage of the SDF architecture for radix R FFT. . . .	27
3.3	Data flow of the SDF architecture for radix-2 FFT. . . .	28
3.4	Data flow of the SDF architecture for radix-3 FFT. . . .	29
3.5	High level block diagram of the reconfigurable SDF architecture.	31
3.6	Reconfigurable radix-2 stage of the SDF architecture. . . .	32
3.7	Padder of the radix-2 stage.	32
3.8	Reconfigurable radix-2/3 stage of the SDF architecture. . .	33
3.9	Radix-2/3 Padder.	34

3.10	Reconfigurable radix-2/3/5 stage of the SDF architecture.	35
3.11	Radix-2/3/5 Preadder.	37
3.12	Twiddle factor unit.	38
3.13	Rotator.	39
3.14	Phase and delay counters.	42
3.15	Stage control logic.	43
3.16	Data and control flow of the stage.	44
3.17	Top level: AXI4-Lite configuration interface, shadow and active registers, chain of stages and AXI4-Stream ports. .	45